

目录

1.	软件的使用	3
1.1	建立工程	3
1.2	打开工程	8
1.3	建立工程编译文件	9
2.	程序烧写	16
2.1	FPGA 程序烧写	16
2.1.1	JTAG Download 模式	17
2.2.2	FLASH 下载模式	20
2.2	CH32V103 程序烧写	24
2.2.1	安装固件驱动	24
2.2.2	烧写 U 盘固件设置	26
2.2.3	程序在线调试烧写-断电后, 程序会丢失	28
2.2.4	程序固化烧写-断电后, 程序不会丢失	30
3.	实例	32
3.1	点亮 LED 灯	32
3.1.1	实验简介	32
3.1.2	硬件设计	32
3.2.3	程序设计	33
3.2.4	实验结果	45
3.2	实例—舵机	46
3.2.1	实验简介	46
3.2.2	硬件设计	46
3.2.3	程序设计	47
3.2.4	实验结果	49
3.3	实例—电机	49
3.3.1	实验简介	49
3.3.2	硬件设计	50

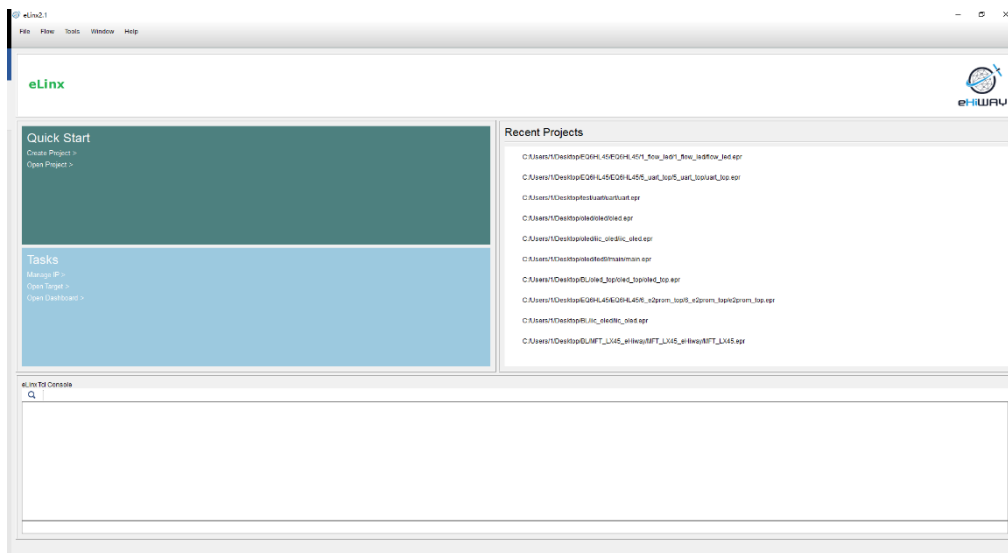
3.3.3	程序设计	52
3.3.4	实验结果	54
3.4	实例—OLED	54
3.4.1	实验简介	54
3.4.2	硬件设计	55
3.4.3	程序设计	56
3.4.4	实验结果	59

1. 软件的使用

软件的使用主要针对工程的建立、打开已有的工程和编译文件的创建；若需打开已有的工程，可直接跳过1.1即可。

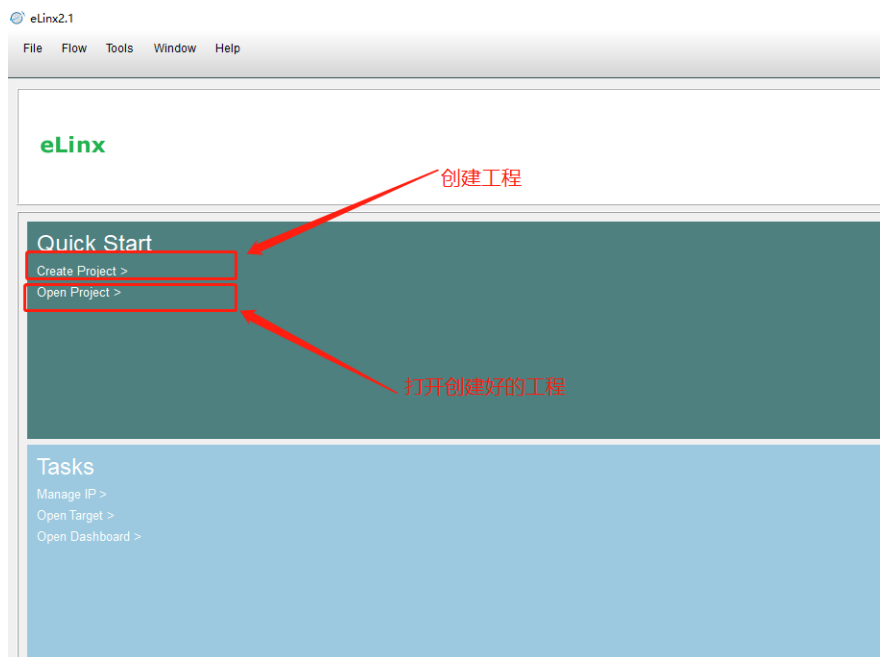
1.1 建立工程

1) 首先双击eLinx软件进入到软件的主界面。

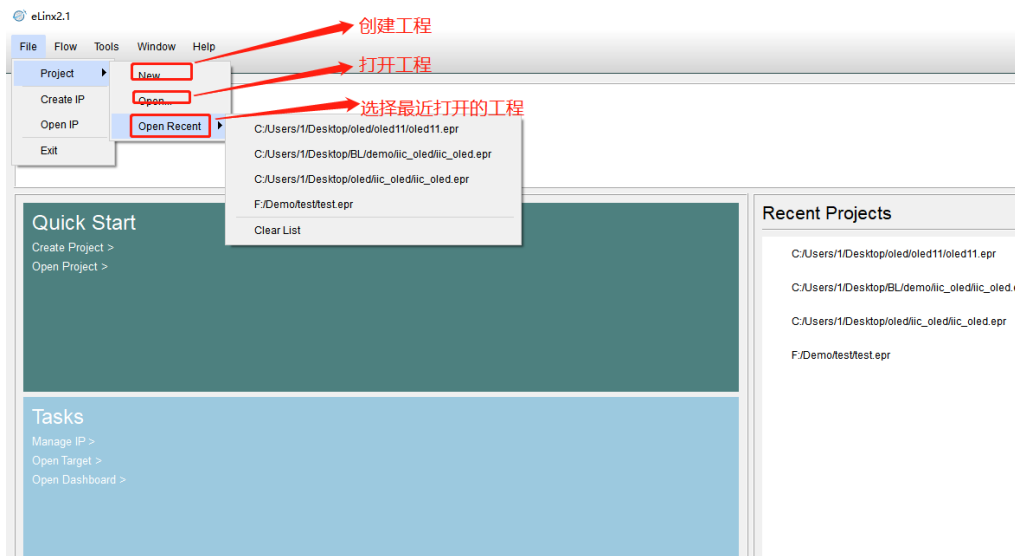


2) 创建工程

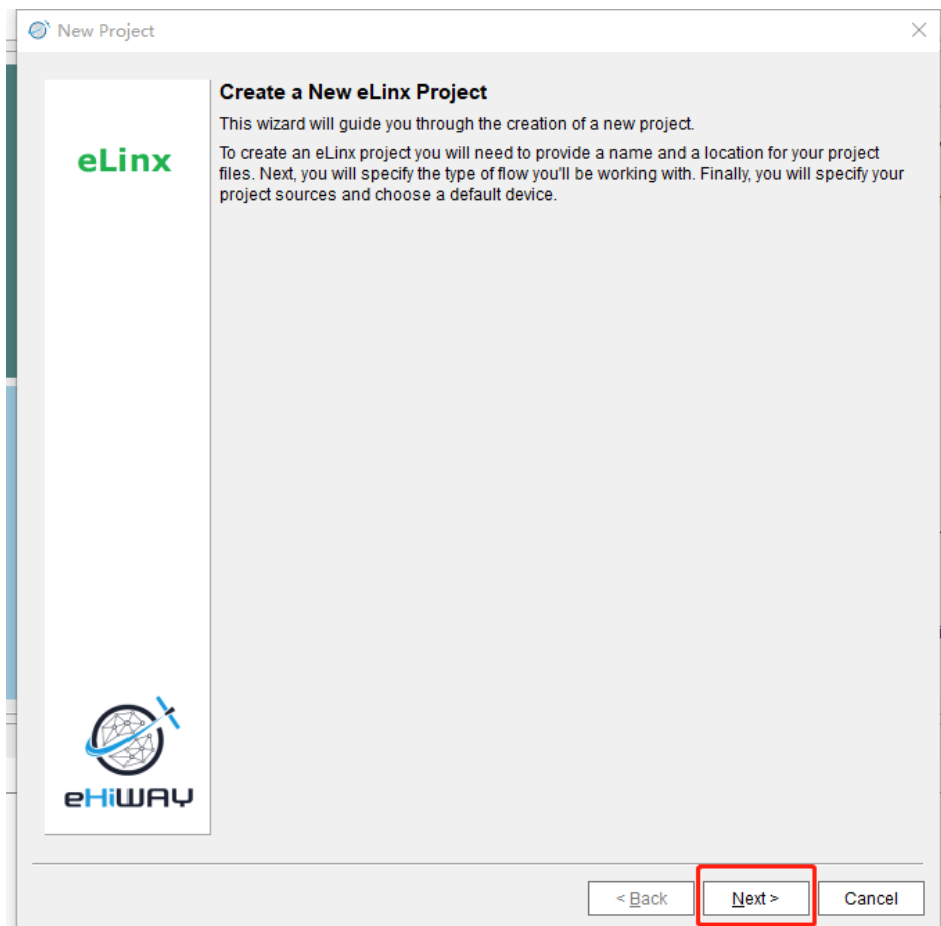
方法一： 点击Quick Start界面中的Create Project的选项



方法二：在菜单栏选项中，点击File-Project-New创建新的工程。

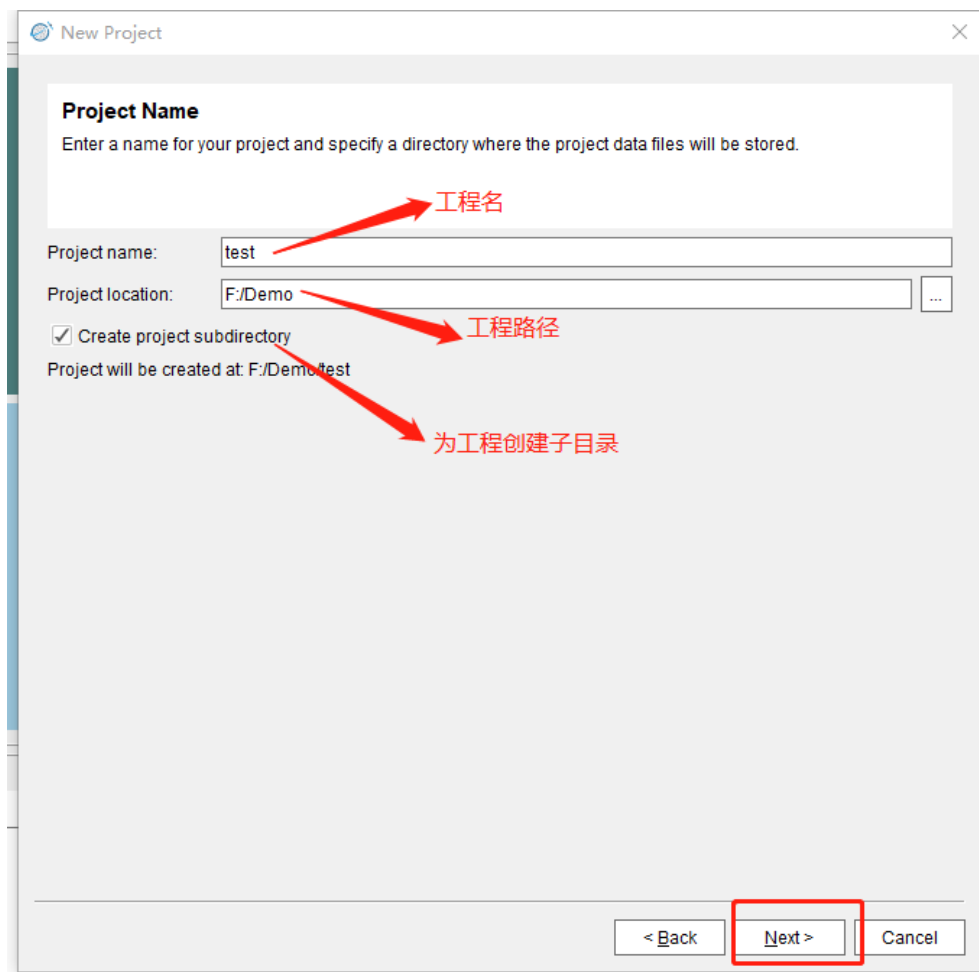


3) 创建工程后进入以下界面，点击Next。



4) 在以下界面Project Name栏输入工程的名字，在Project Location栏选择工程的路径，Create project subdirectory选项来确定需要不需要为工程路径创建

一个子目录，一般都需要勾选上。然后，点击Next。



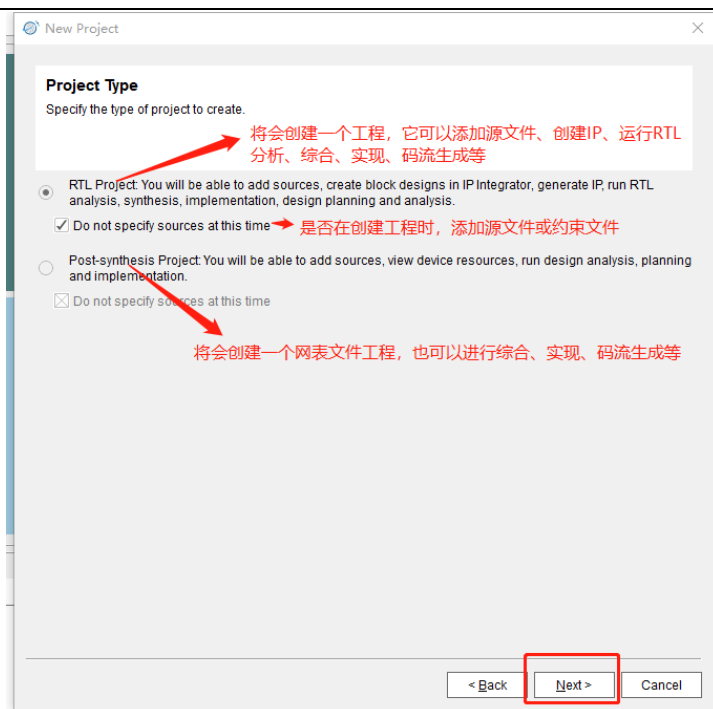
5) 进入以下界面，其中：

RTL Project: 将会创建一个工程，它可以添加源文件，创建IP，运行RTL分析、综合、实现、码流生成等。

Post-synthesis Project: 将会创建一个网表文件工程，也可以进行综合、实现、码流生成等。

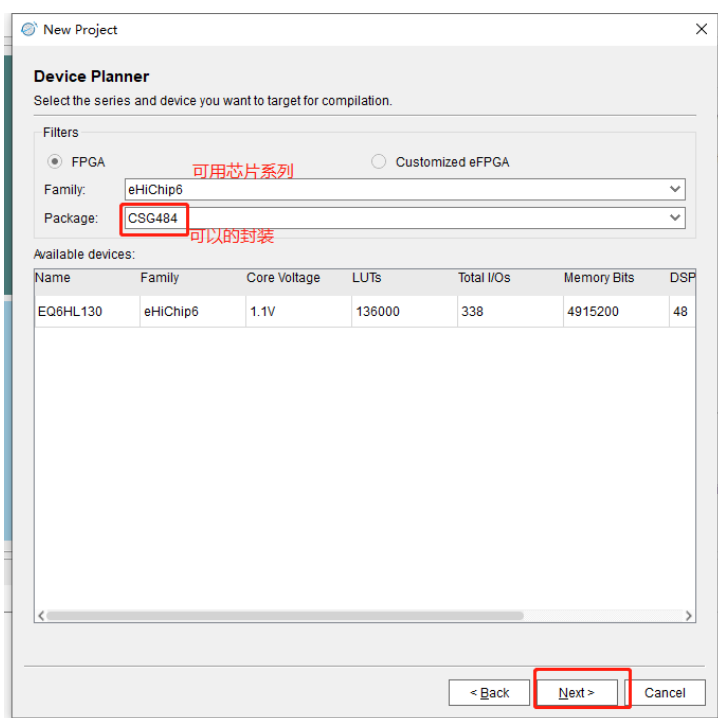
Do not specify sources at this time: 选项来确定您需要不需要在创建工程时就添加源文件或约束文件，如果勾选上，则不会在创建时添加源文件和约束文件(点击下一步时就不会出现添加源文件窗口，而是会直接到器件选择窗口)，但您可以在后面介绍的工程Source窗口右键菜单添加。

选择RTL Project，并选中Do not specify sources at this time,点击Next。

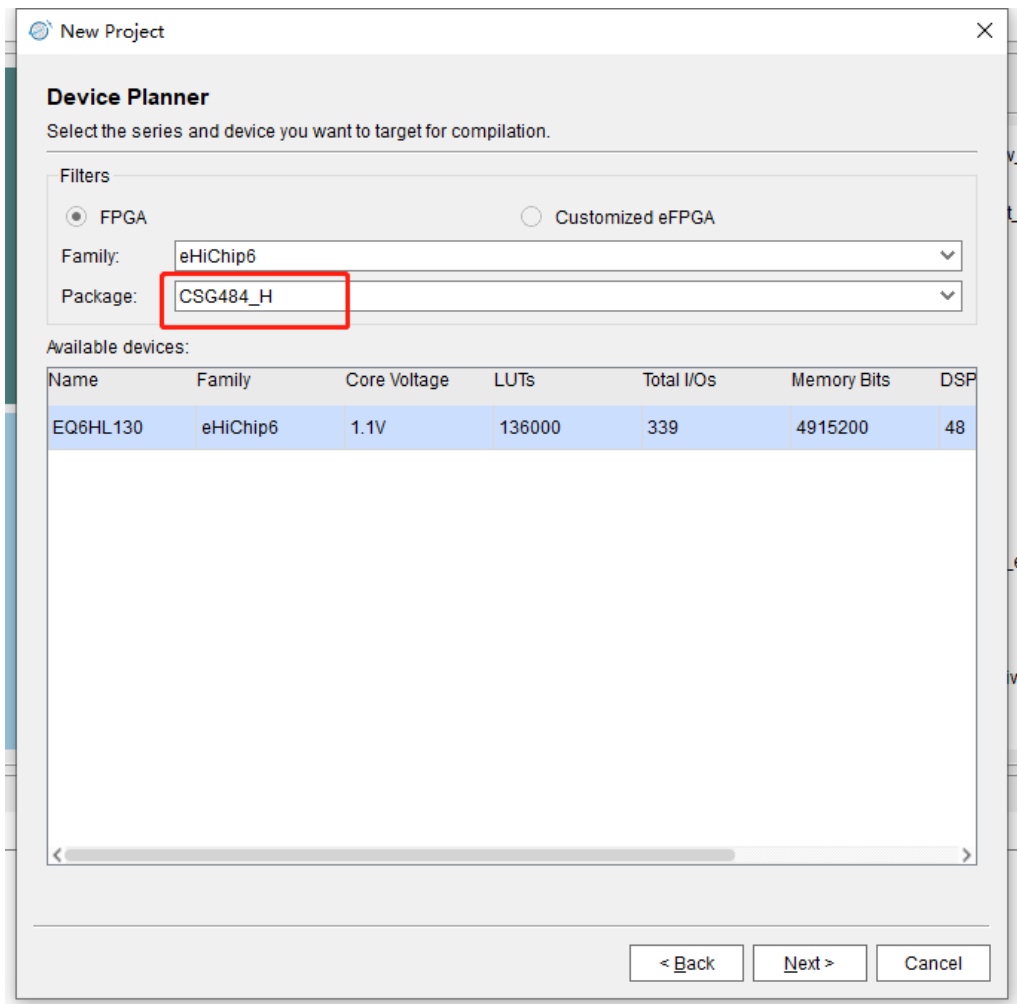


6) 以下界面可以选择准备使用的芯片，点击Next。

其中，Family 下拉框表示可用的系列；Package 下拉框表示可用的封装 (Customized eFPGA 没有 Package 值)；Available devices 列表显示了Family系列下封装是 Package 的可供选择的所有 devices，并且显示了对应 device 的一些资源信息。



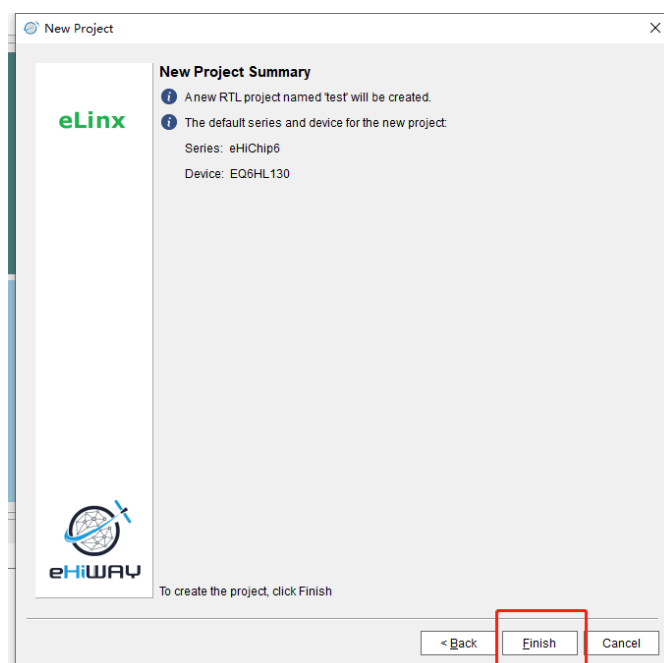
本步骤下需要检查用户使用芯片，若使用的是eHiChip6系列的EQ6HL130LL-2CSG484封装芯片时，需注意该型号批次，2146批次（不含）之前的芯片需选择CSG484封装，如上图红框所示；如果是2146批次（含）之后的芯片需要选择CSG484-H封装，如下图红框所示。



芯片产品批次号查看位置见芯片丝印，如下图红框内。



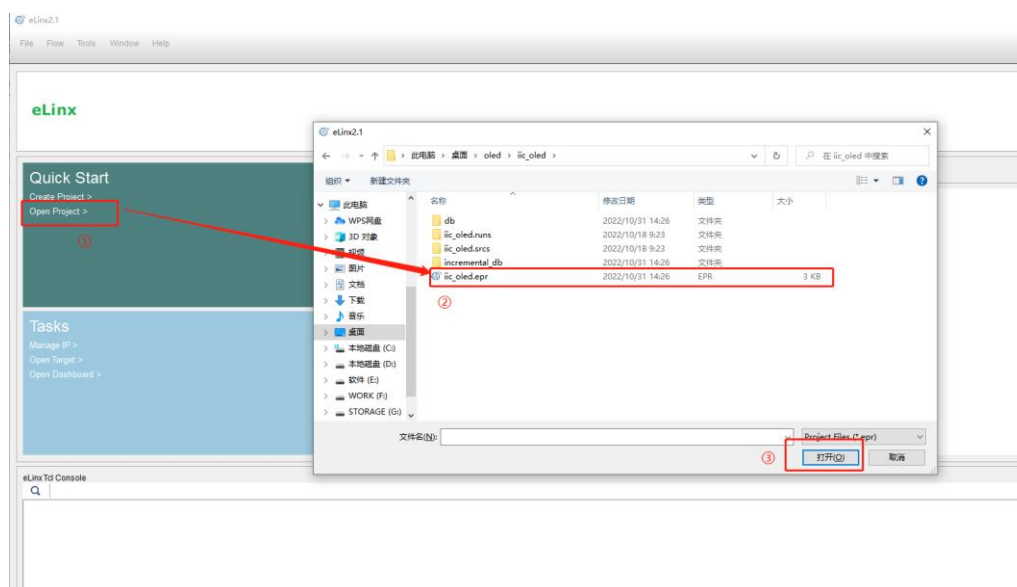
7) 进入下面界面，点击Finish。



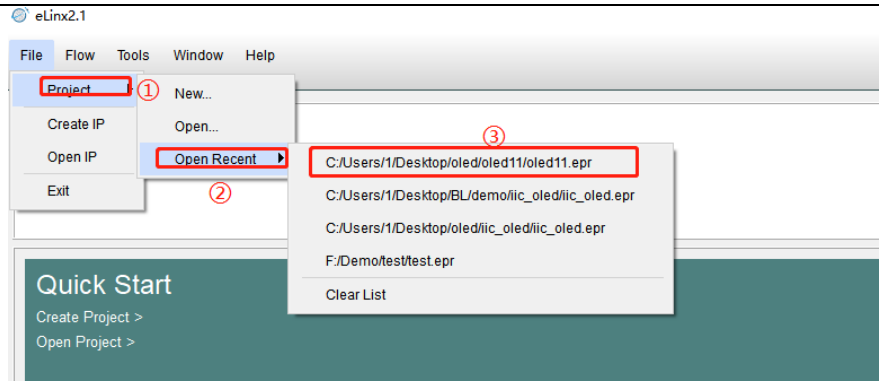
1.2 打开工程

打开软件之后，会进入以下 Quick Start 主界面，在 Quick Start 页面，打开工程有三种方法：

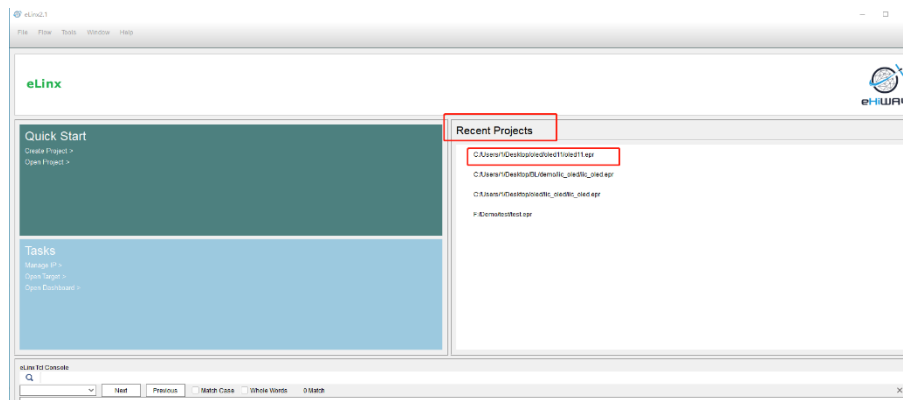
方法一：点击 Quick Start 处的Open Project，在弹出的选择工程对话框中，选择.epr 类型的文件，即可打开相应工程；



方法二：Recent Projects 列表中，点击一个最近打开的工程，即可打开相应工程。



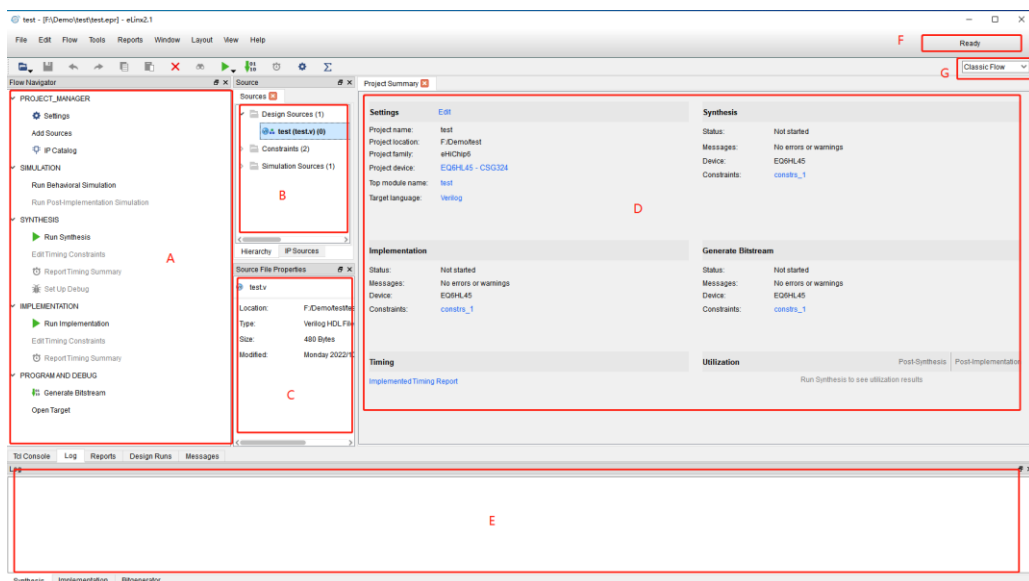
或者在Quick Start 主界面，Recent Projects栏下打开。



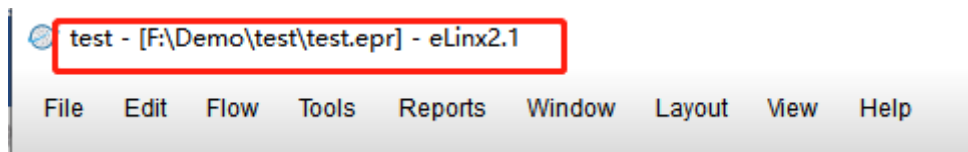
方法三：在 eLinX Tcl Console 窗口输入框中输入命令” open_project 工程 epr 全路径”，然后回车，即可打开相应工程。

1.3 建立工程编译文件

1) 新建工程完成后，将会进入以下界面：

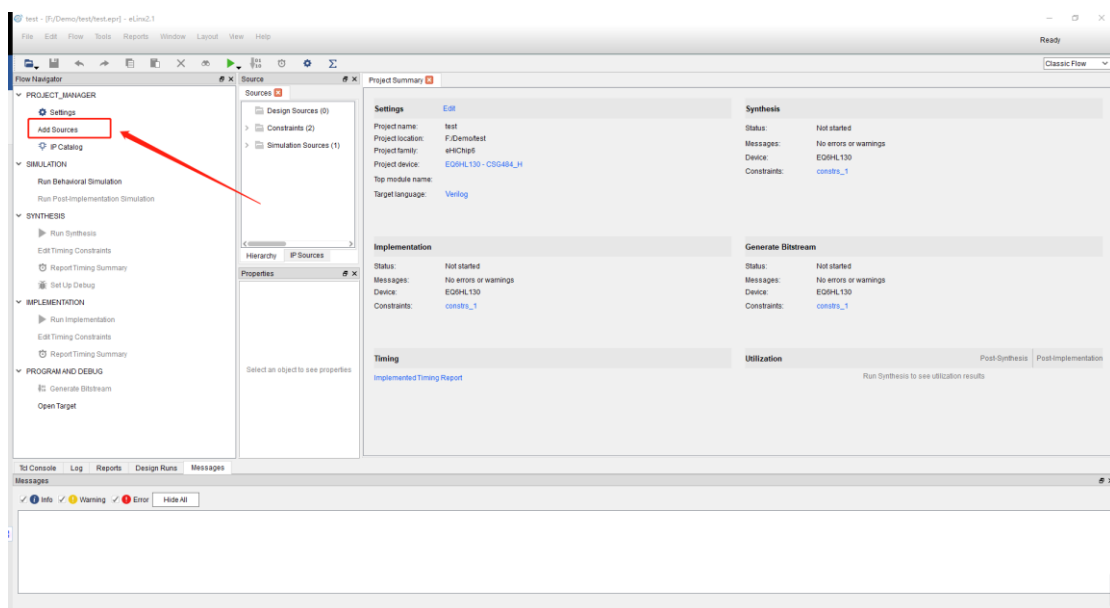


- ① 标题显示了当前打开工程名称和所在目录

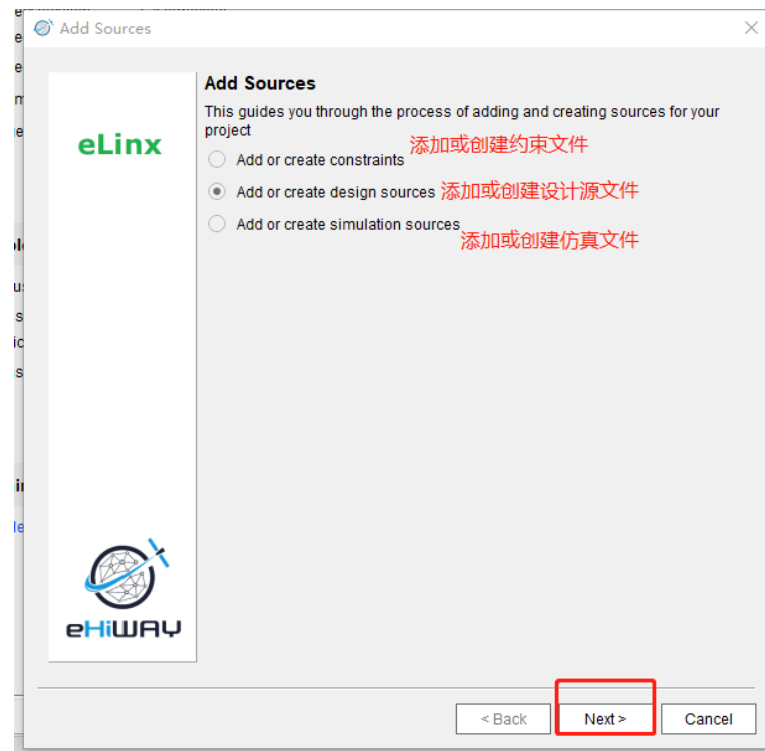


- ② A 区域是 Flow Navigator,它包含了 eLinx EDA 工具的整个流程: Project manager、Simulation、Synthesis、Implementation、Program and debug。
- ③ B 区域是工程所有文件显示窗口,分为三个部分:工程源文件 Design Sources、约束集合 Constraints、仿真源文件集合 Simulation Sources。
- ④ C 区域是属性展示窗口,它可以展示文件、各个 runs、I/O Port、Package Pin 等等的属性。
- ⑤ D 区域是主要工作区域,打开的文件、I/O Planning、Floor Planning 等等窗口都会显示在这个区域。
- ⑥ E 区域包含了 Tcl Console、Log、Reports、Design Runs、Messages 等窗口。
- ⑦ F 区域显示了当前 run 的当前状态。
- ⑧ G 区域显示了当前的流程类型,有 Classic Flow 和 Evaluation Flow 可供选择

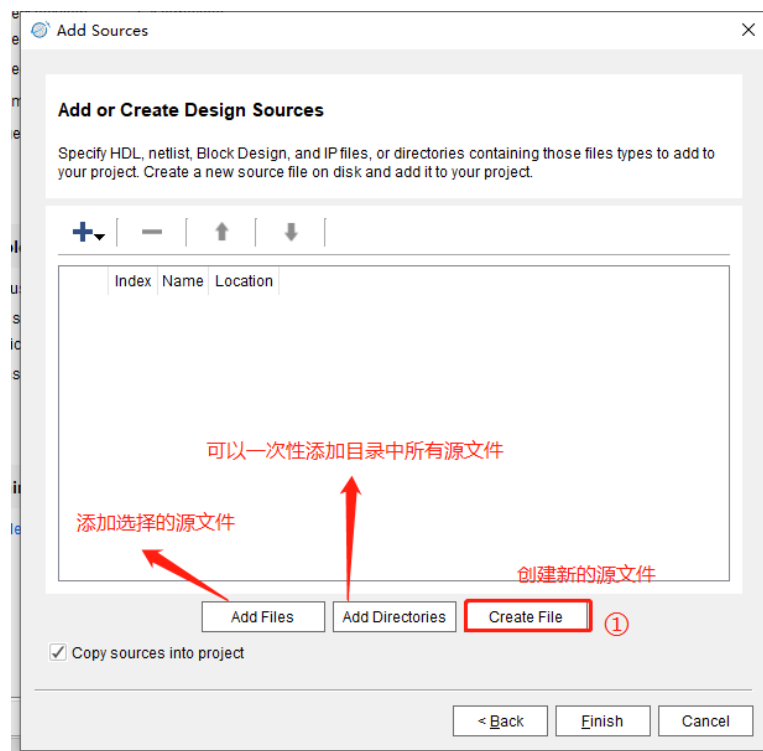
2) 进入主界面之后双击Flow Navigator栏Add Sources来添加源文件。

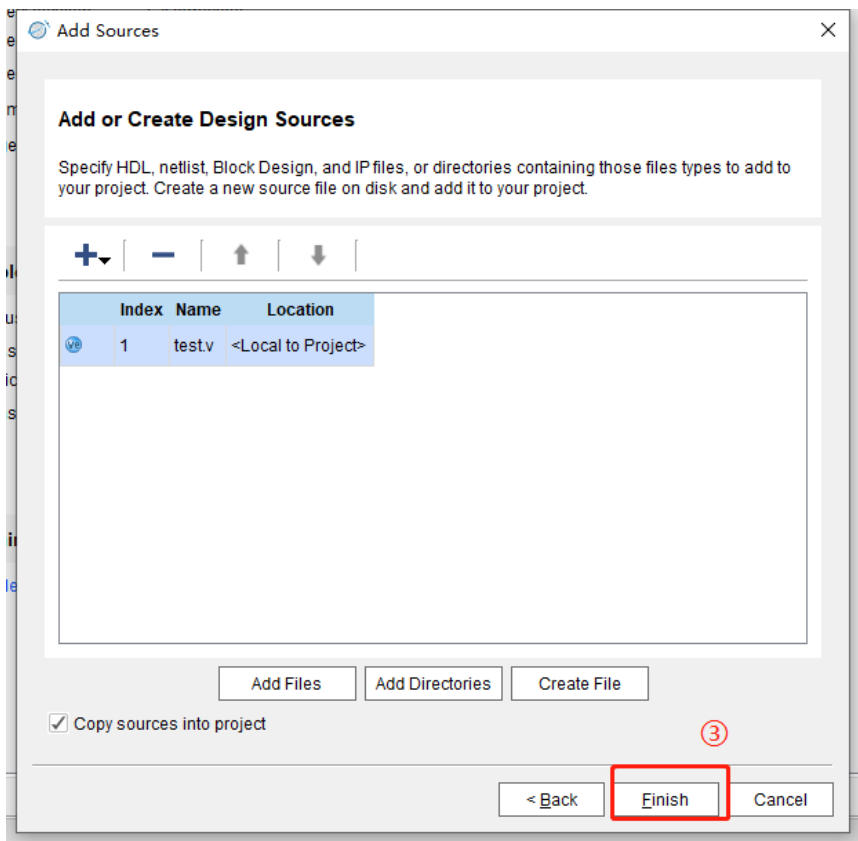
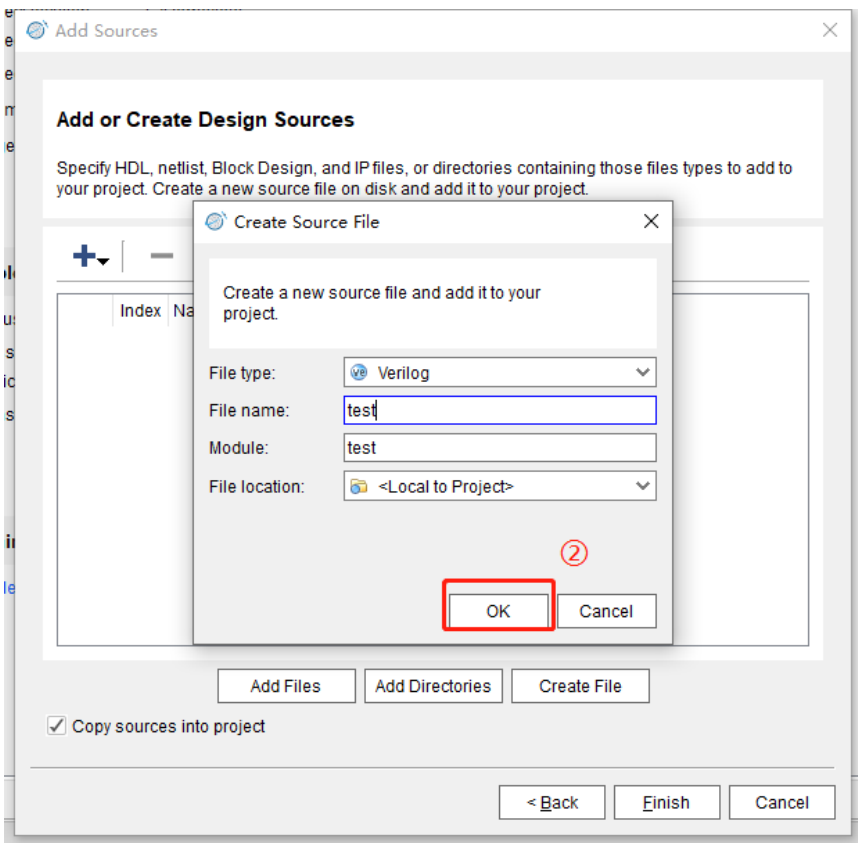


3) 选中Add or create design sources，点击Next。



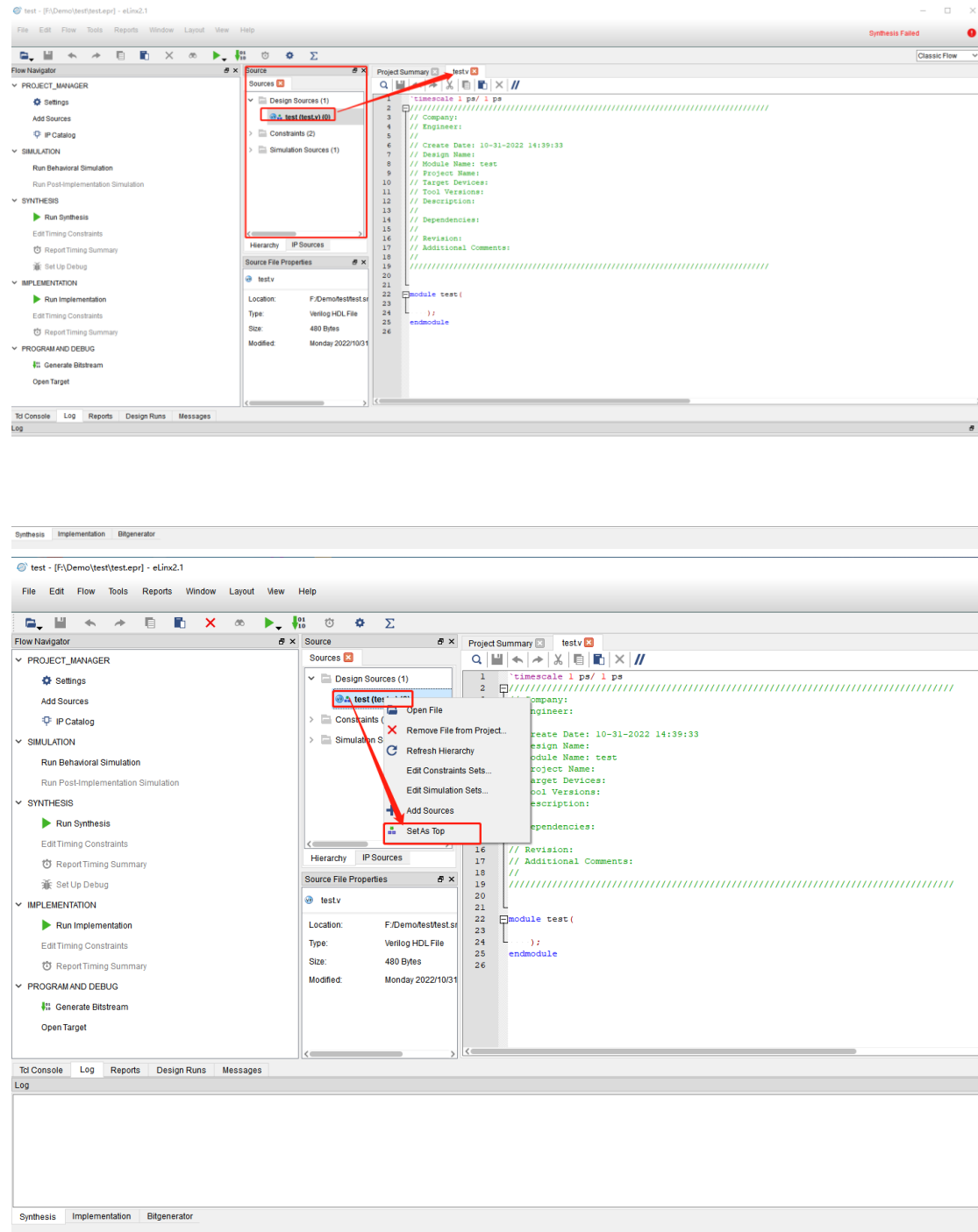
4) 点击Create File，在File type栏选择使用的硬件语言，在File name栏输入源文件名字点击OK，点击Finish。（其中：Copy sources into project 表示添加的源文件是否需要拷贝到工程源文件目录中）



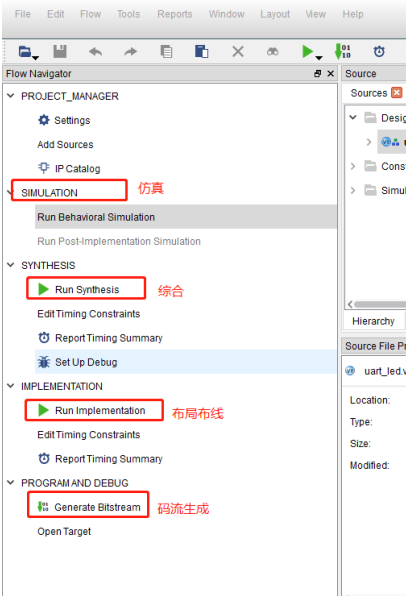


5) 在Sources栏找到创建的.v或.vhd文件并双击，即可编辑自己的程序，当创建

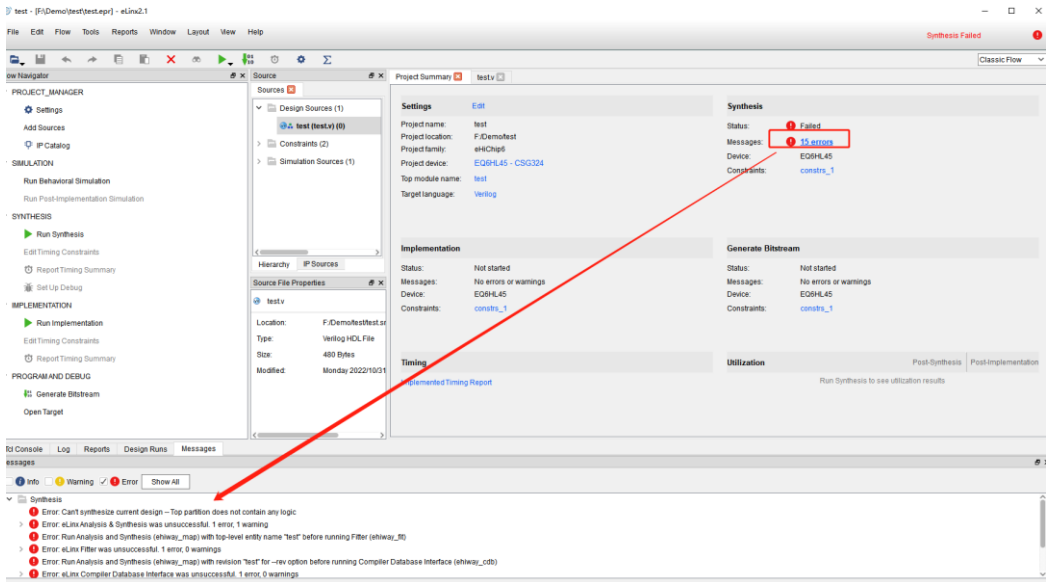
多个源文件时，可以右击需要设置为头文件的文件，点击Set As Top来设置为顶层文件。



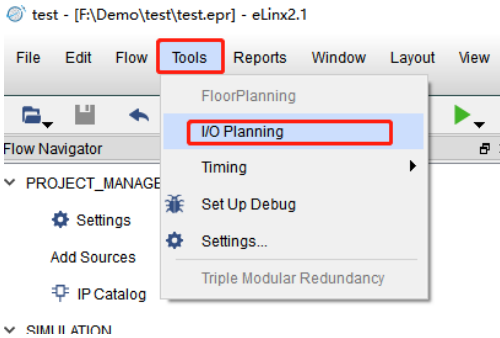
- 6) 当程序编写完成后点击保存开始综合，点击Run Synthesis，在主界面可以查看是否出现错误。



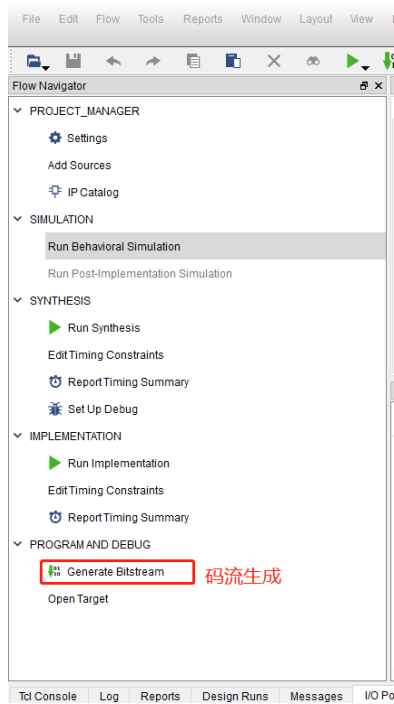
若在编译过程中，出现报错，点击报错数量，即可查询错误信息



7) 综合完成之后开始绑定管脚，点击Tools/ IO Planning。



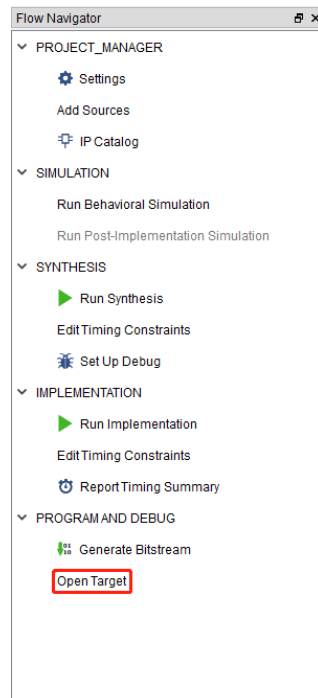
8) 这里下面可以进行管脚的绑定，同时可以清晰的看到管脚绑定的情况，点



2. 程序烧写

2.1 FPGA程序烧写

- 1) 码流生成之后，双击Open Target准备下载到芯片里。

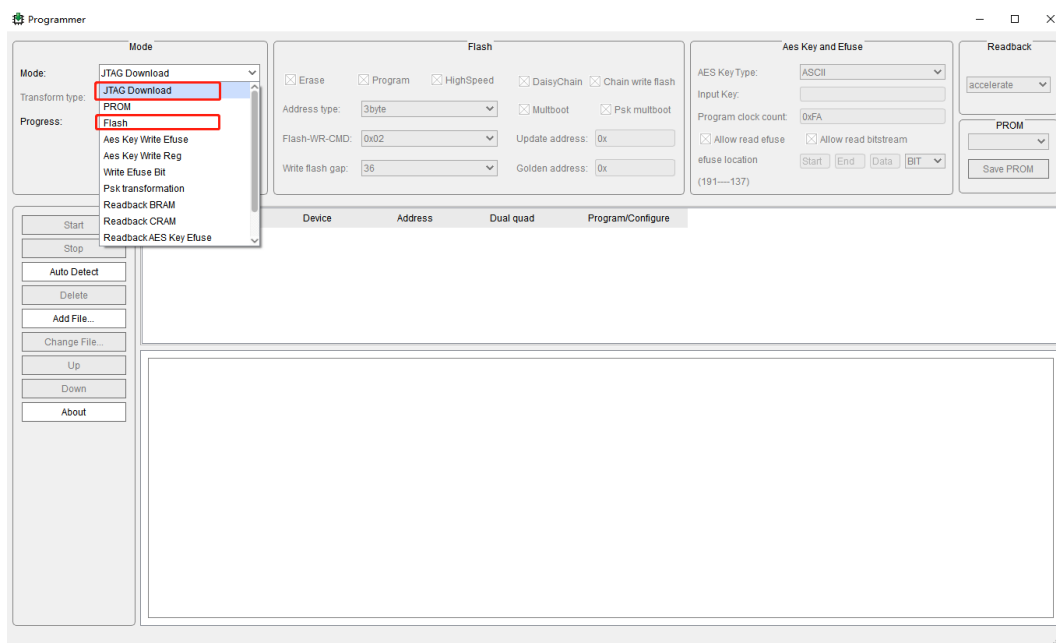


- 2) 这里可以在Mode栏选择JTAG或FLASH下载模式。根据不同的下载模式需要选择不同的文件。

①、JTAG Download模式: 使用 USB JTAG 下载码流数据到 FPGA 中。

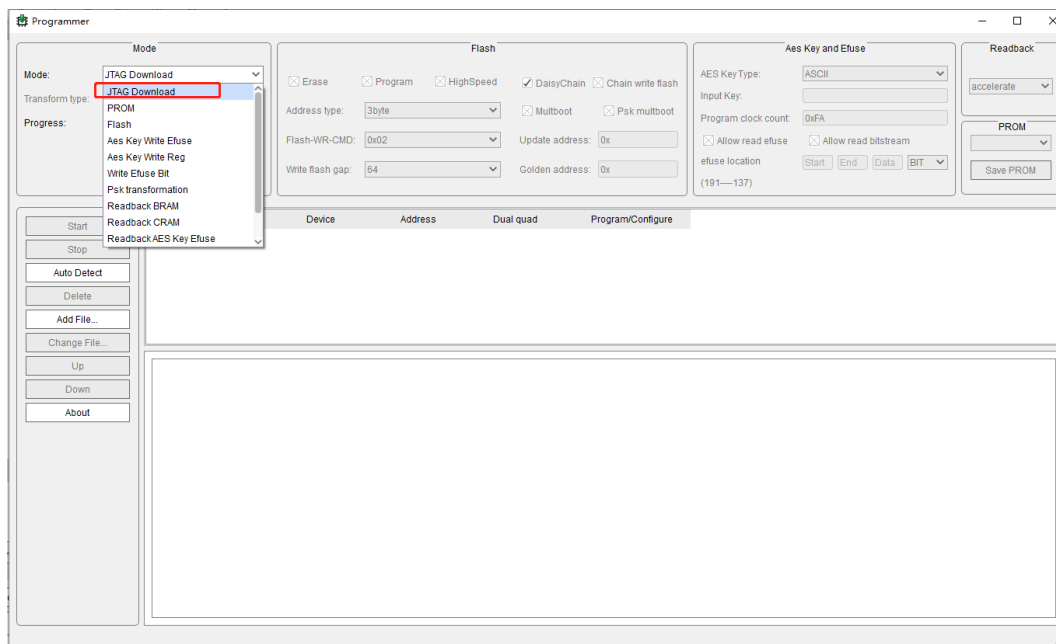
(注: 断上电后, 需重新下载程序)

②、Flash模式: 把码流数据写入 Flash。(注: 断上电后, 会自动从芯片中加载已下载好的程序)



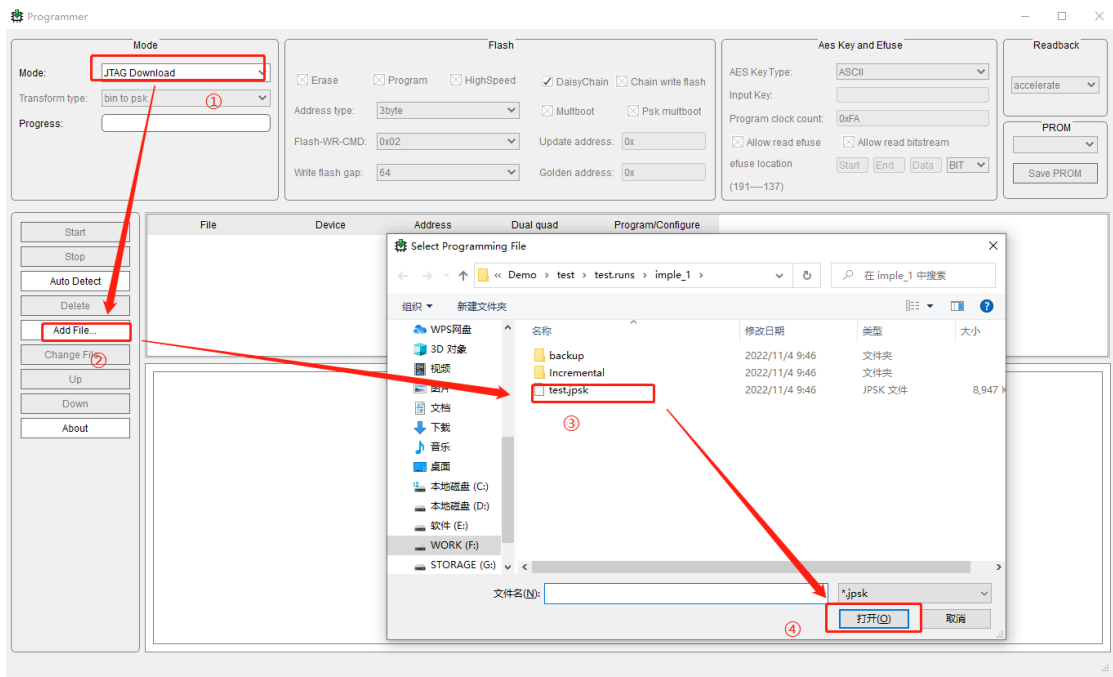
2.1.1 JTAG Download 模式

1) 在Mode栏, 选择JTAG Download下载模式

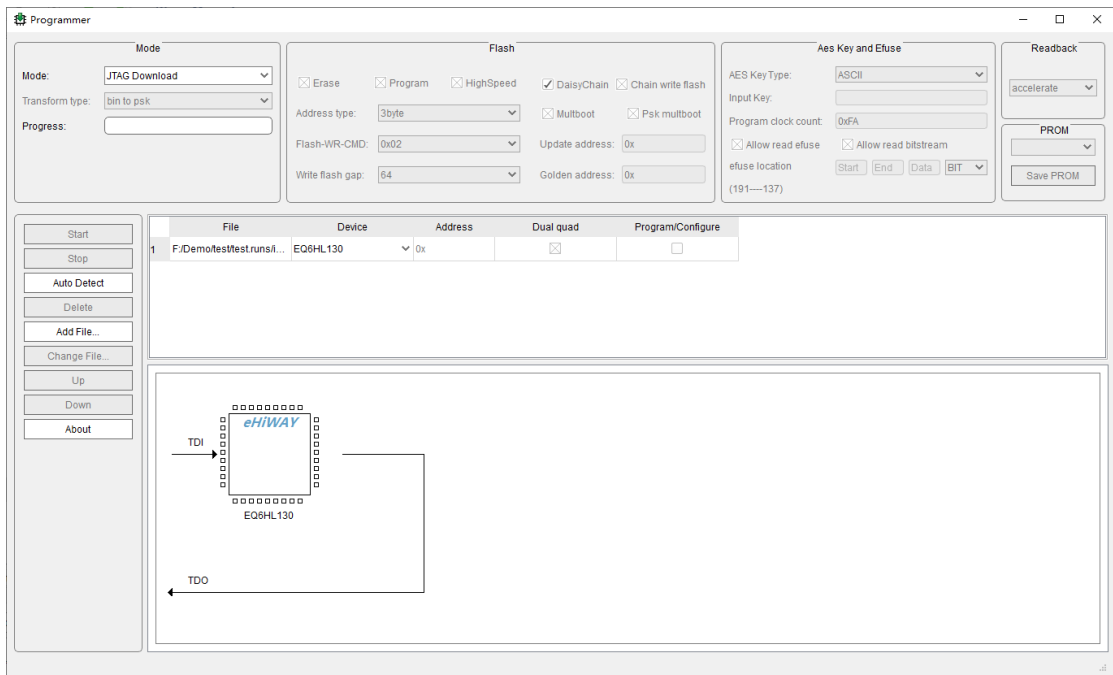


2) 如下图所示, 当选择JTAG下载模式时, 点击Add File, 选择生成的码流文

件，文件后缀名为.jpsk文件。

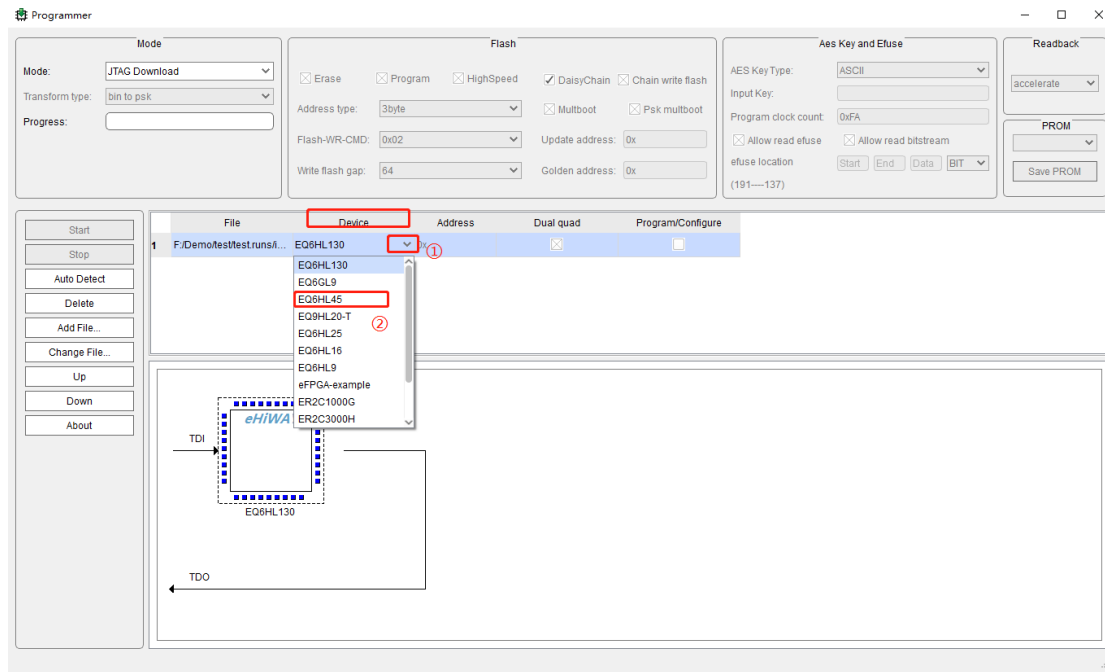


3) 当选择好下载模式和添加好要加载的文件后，如下图所示（注：系统芯片器件型号为EQ6HL130）

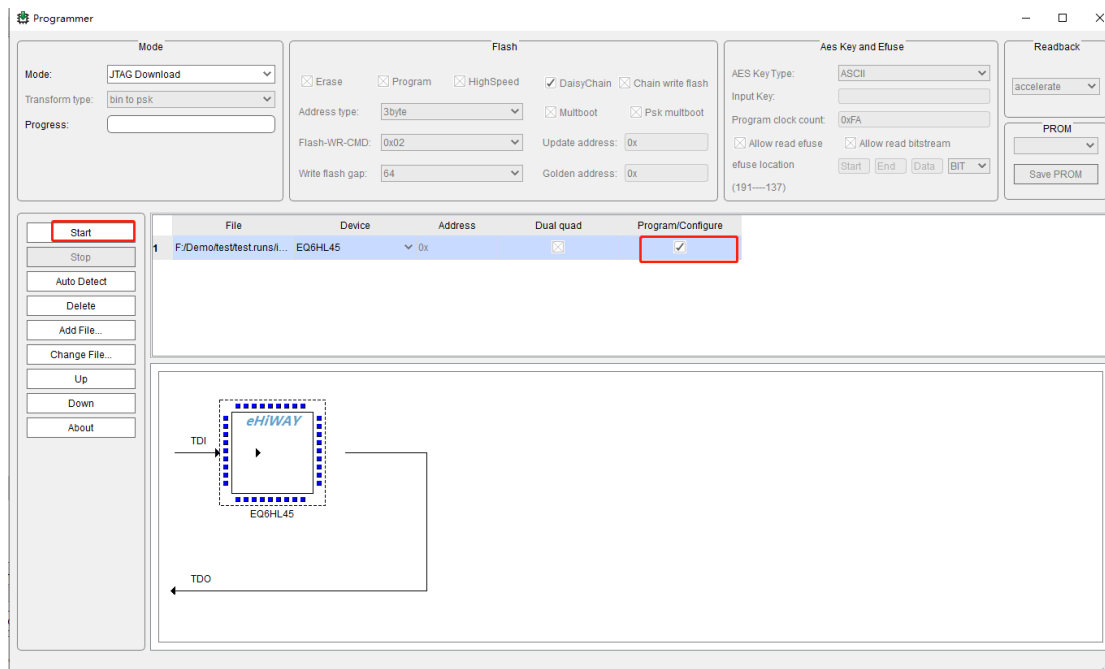


4) 本例程所选择的芯片器件型号为EQ6HL45，需对芯片器件型号进行选型，在Device栏，选择下拉框选择对应的器件型号。

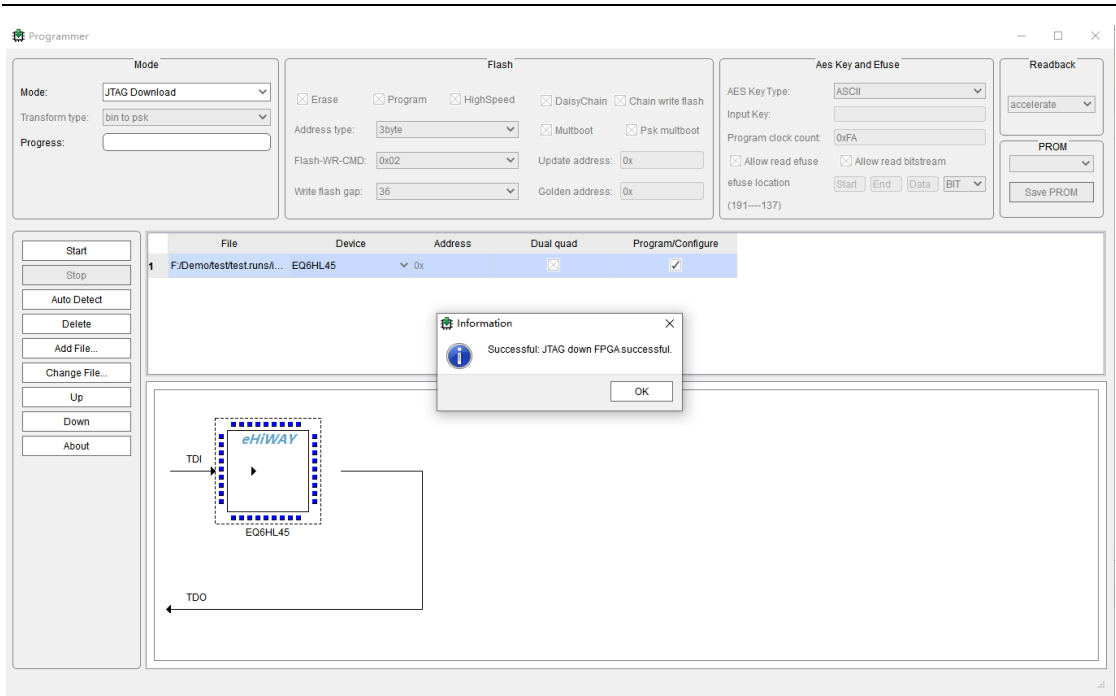
中科亿海微电子科技有限公司(苏州)有限公司



- 5) 选择对应的芯片后，并在后面的小方框内打勾，点击Start即可下载到芯片里。

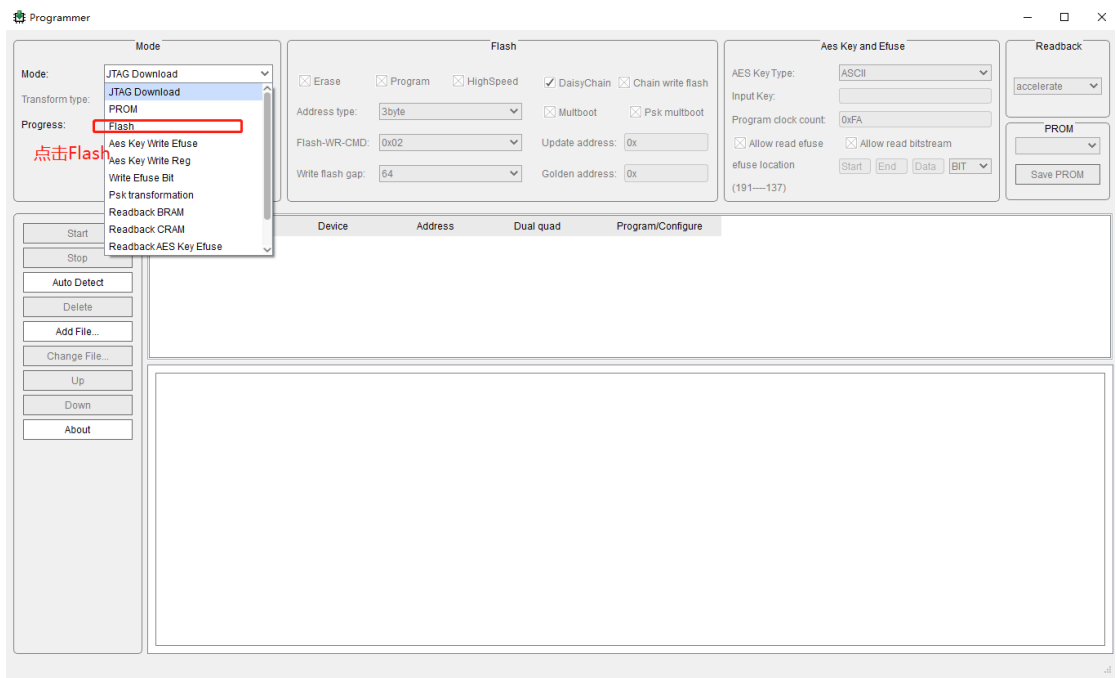


- 6) 如图所示，当我们下载成功后会弹出窗口进行提示，表示你已经成功下载进芯片里。

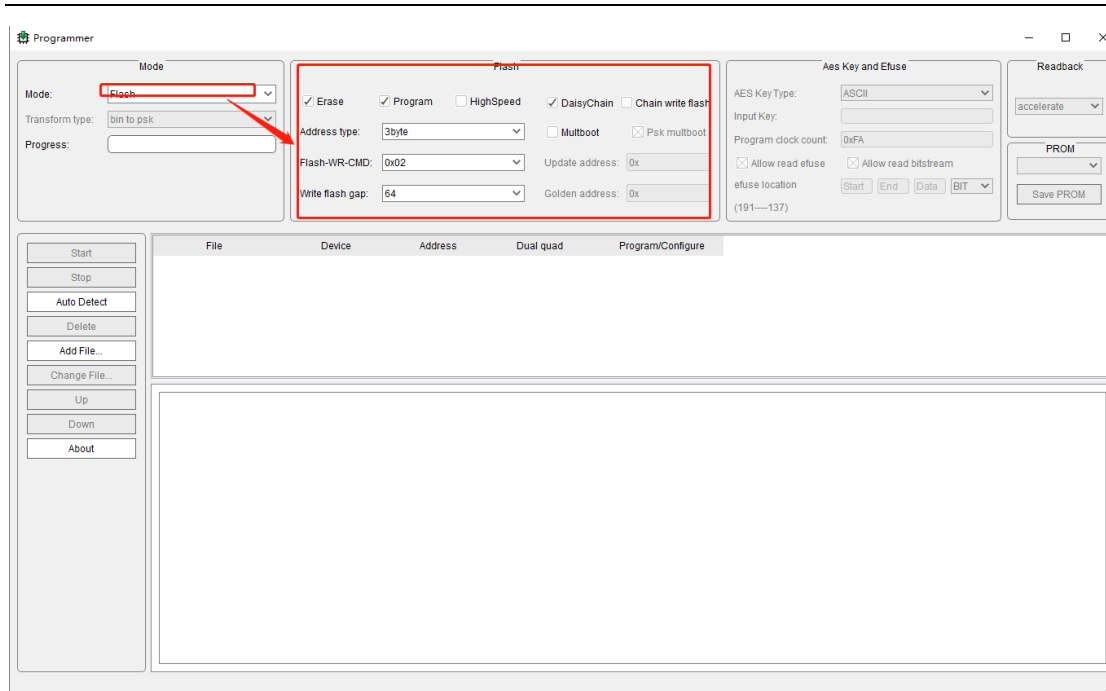


2.2.2 FLASH 下载模式

1) 选择FLASH下载模式后，如下图所示。



2) 当Mode选择了Flash时，Flash栏中的设置才有效，如下图示：



只勾选Program表明直接下载码流到Flash，只勾选Erase，表明只擦除Flash，勾选Program的同时也勾选上Erase，表明先擦除Flash中的数据再下载码流到Flash。

Address type是flash寻址模式，有3byte和4byte两个选项。具体可以参考所使用的FLASH手册中的内容。如SPI命令中地址位的宽度是3byte(24位地址)则选择该项，3byte地址最大支持FLASH容量是256Mbit，而4byte（32位地址）支持FLASH容量是256Mbit~32Gbit。

Flash-WR-CMD是Flash的写命令格式，有0x02和0x12两个选项。（注：一般用3byte写flash都是02，写4byte有些flash需要12指令。）

Write flash gap 是连续两次写 flash 的时间间隔，默认值依据不同芯片类型而不同。（注：在SPI通信中两次发送数据间隔的时钟周期）

HighSpeed 是快慢速下载，默认不勾选是慢速。

Multiboot 是多重启动选项。

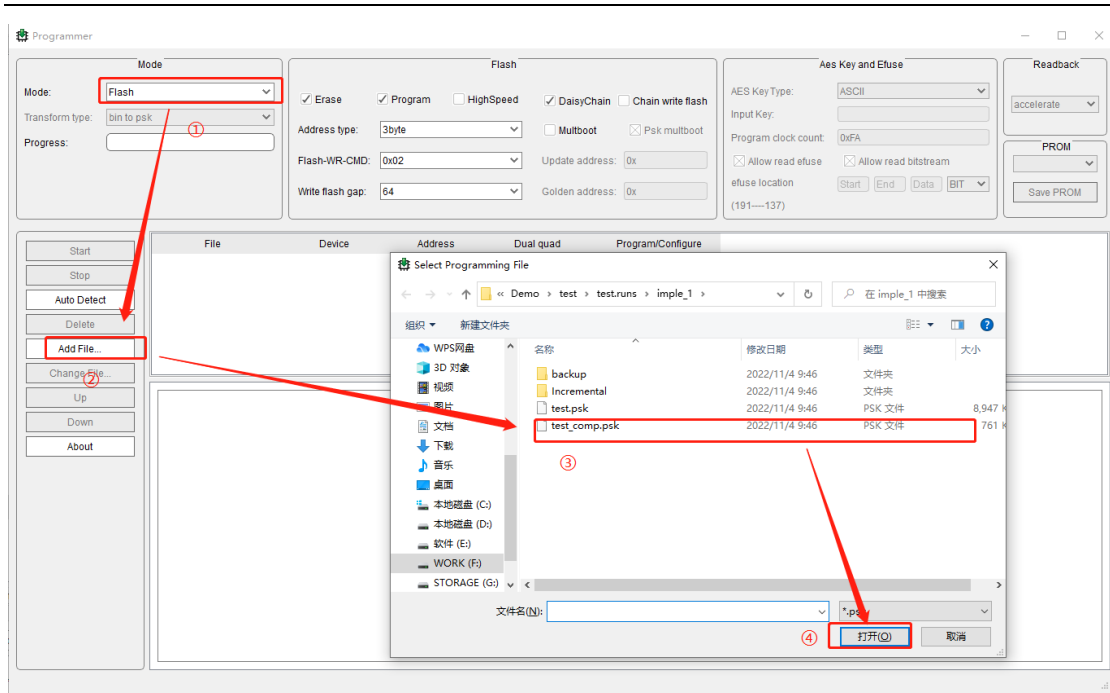
Psk multiboot 指是否指定 psk 多重启动地址。

Update address 是升级地址。

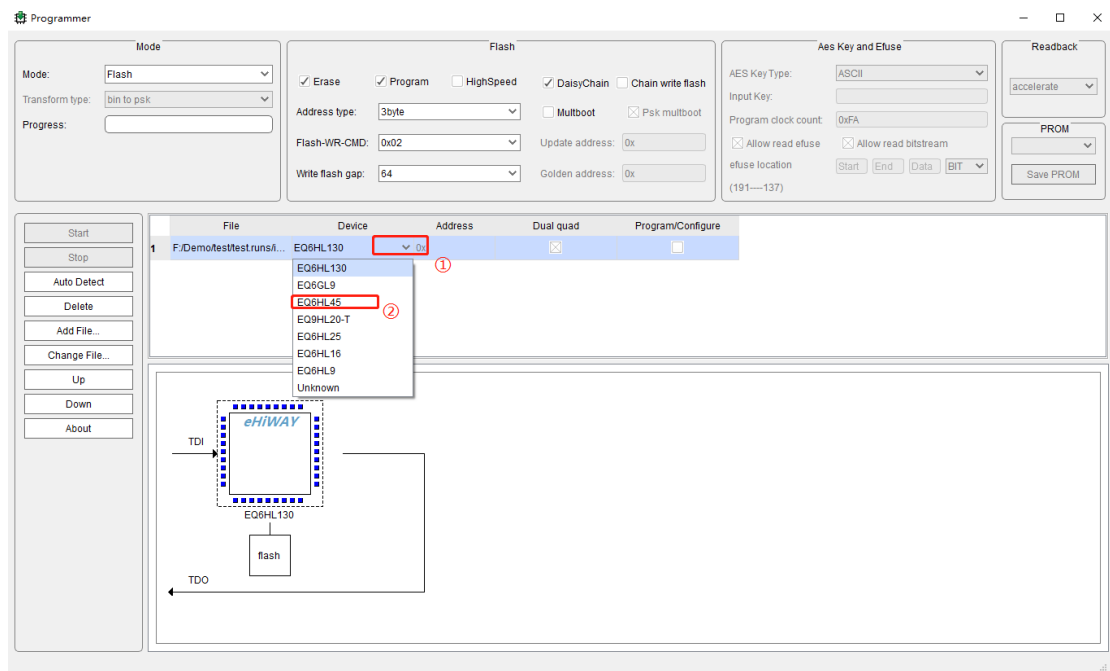
Golden address 是黄金地址。

3) 当选择FLASH下载模式时，注意选择文件后缀名为xxx_comp.psk的文件。

中科亿海微电子技术(苏州)有限公司

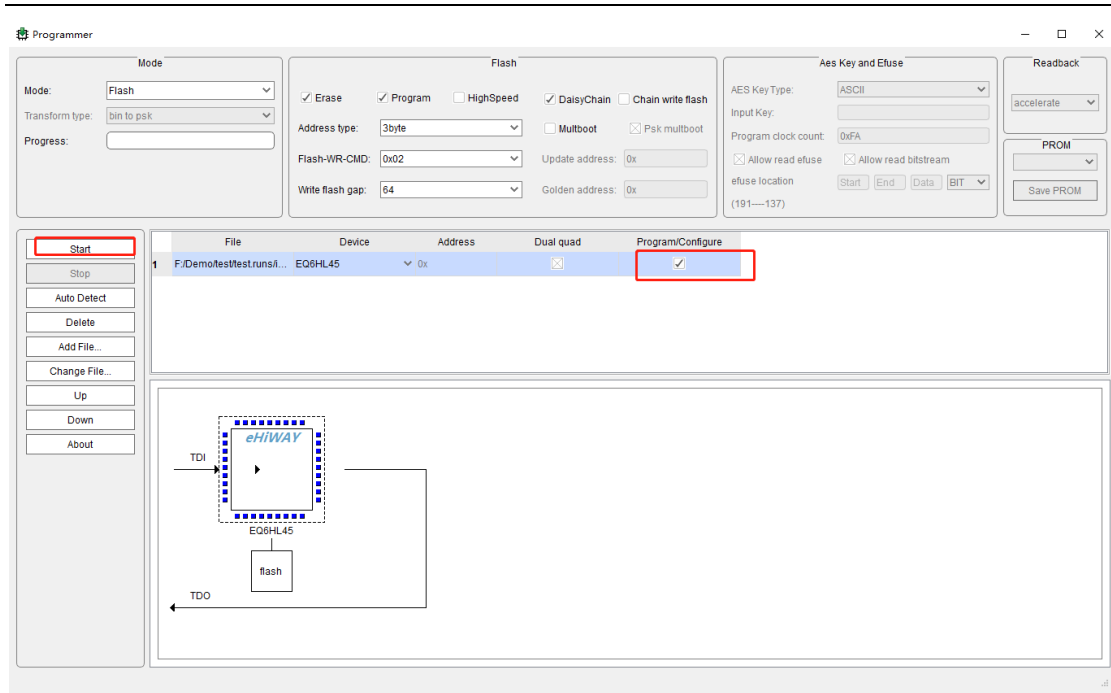


- 4) 本例程所选择的芯片器件型号为EQ6HL45，需对芯片器件型号进行选型，在Device栏，选择下拉框选择对应的器件型号。

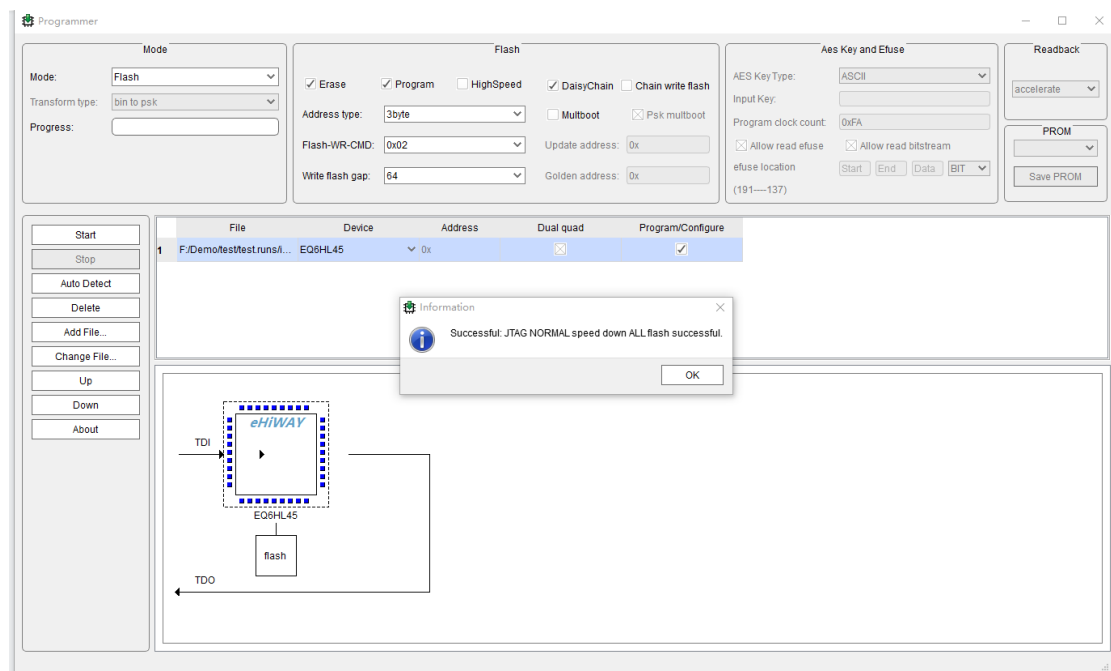


- 5) 选择对应的芯片后，并在后面的小方框内打勾，点击Start即可下载到芯片里。

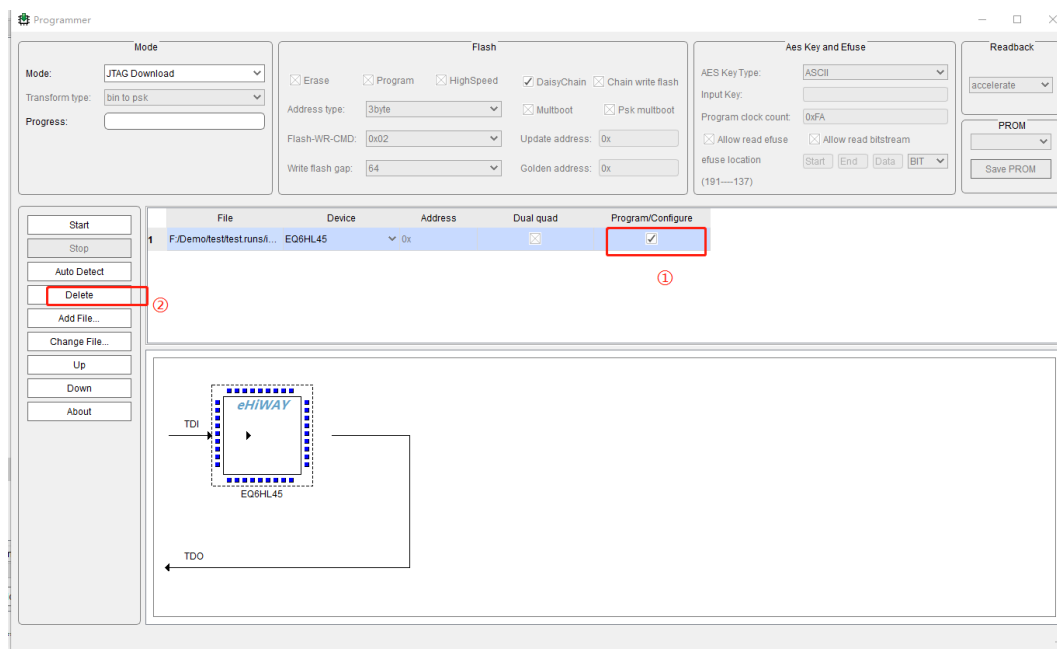
中科亿海微电子科技有限公司(苏州)有限公司



- 6) 如图所示，当我们下载成功后会弹出窗口进行提示，表示你已经成功下载进芯片里。



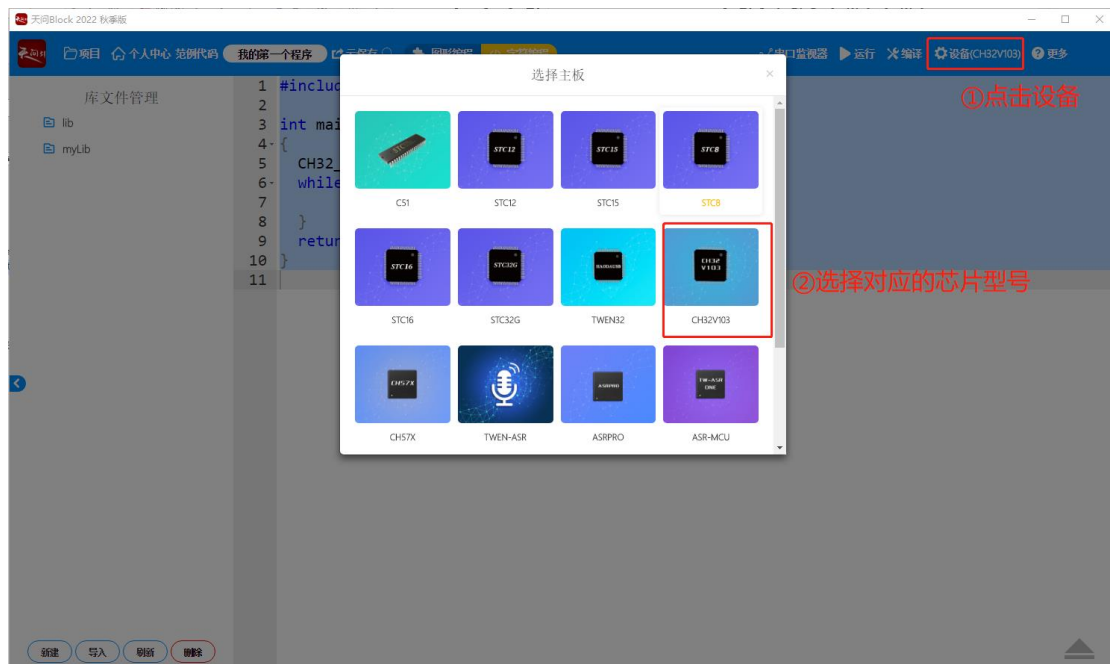
- 7) 需要注意的是，如图所示，当我们已经选定一种模式（如JTAG模式），需要变换为另一种下载模式时，需要选中已经添加进去的文件然后点击Delete选项删去文件，然后在Mode栏更换下载模式重新添加文件。



2.2 CH32V103程序烧写

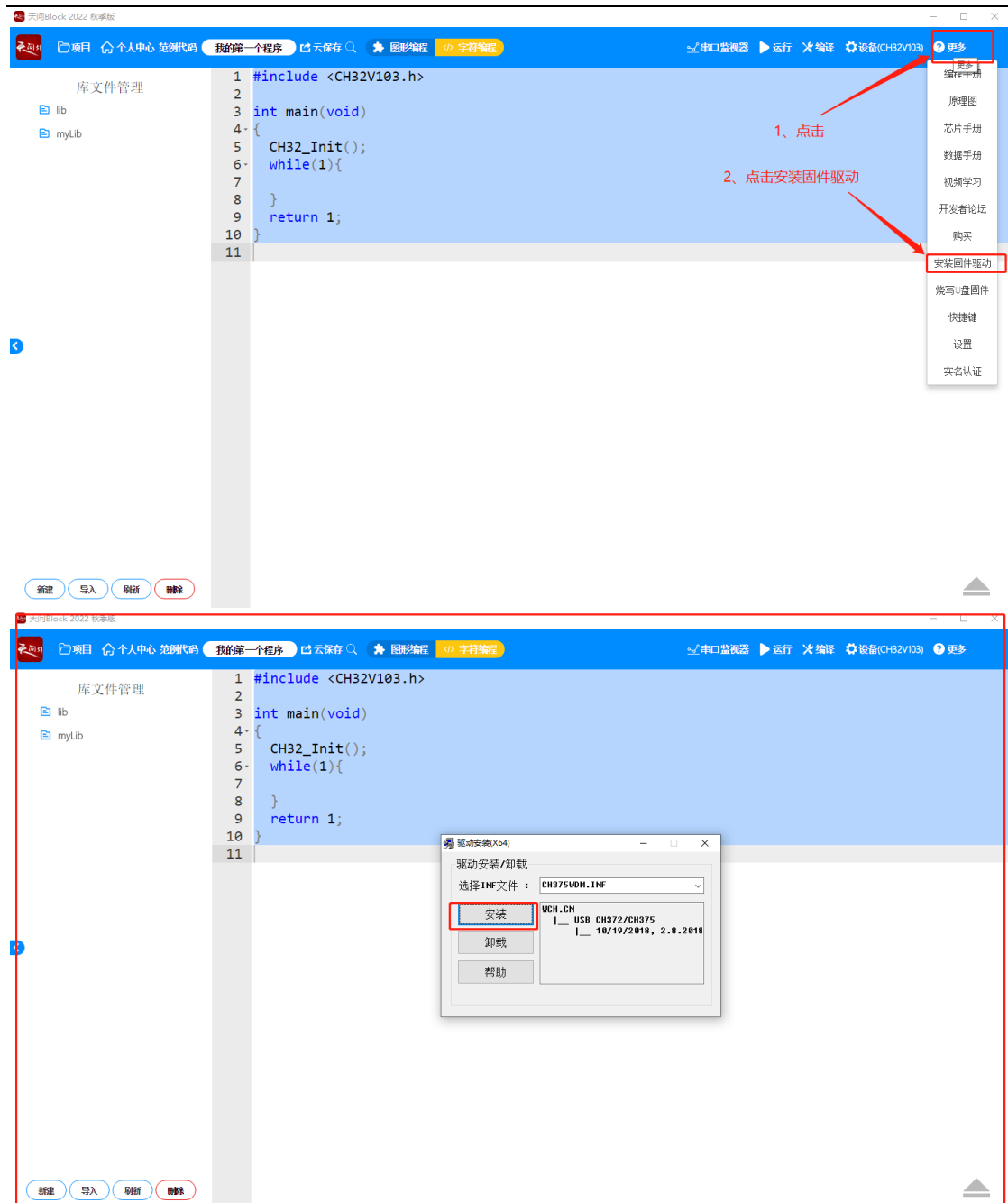
2.2.1 安装固件驱动

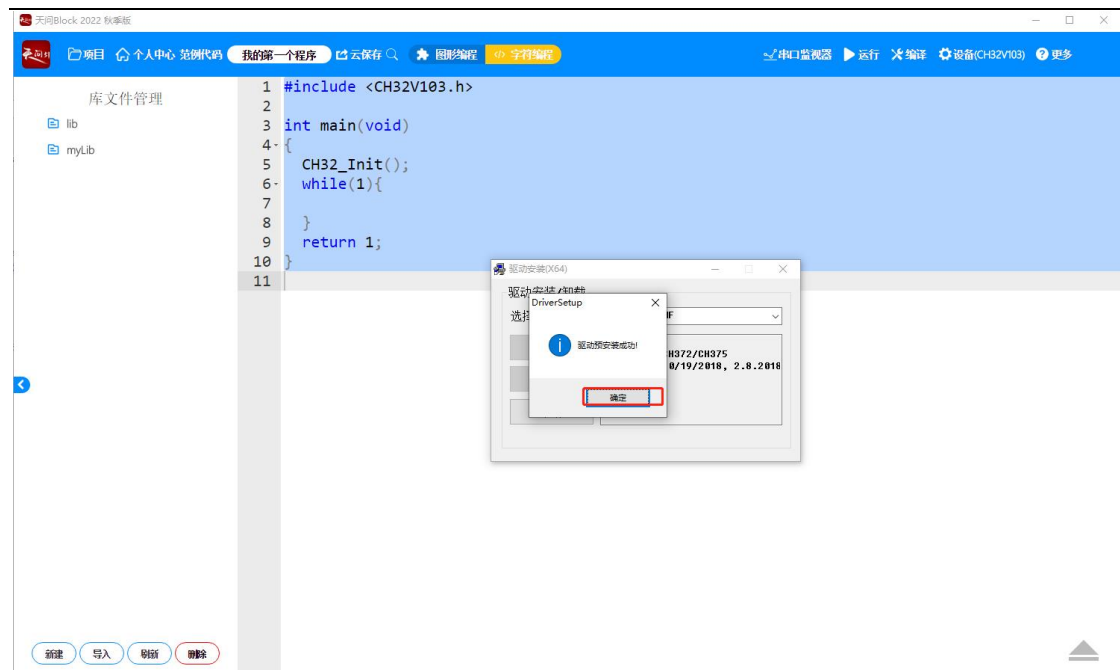
1) 打开天问Block软件后，选择对应的芯片型号，该项目选用的芯片型号为:ch32v103。



2) 点击更多，选择安装固件驱动后，会弹出驱动安装界面，直接点击安装即可。

中科亿海微电子科技有限公司(苏州)有限公司

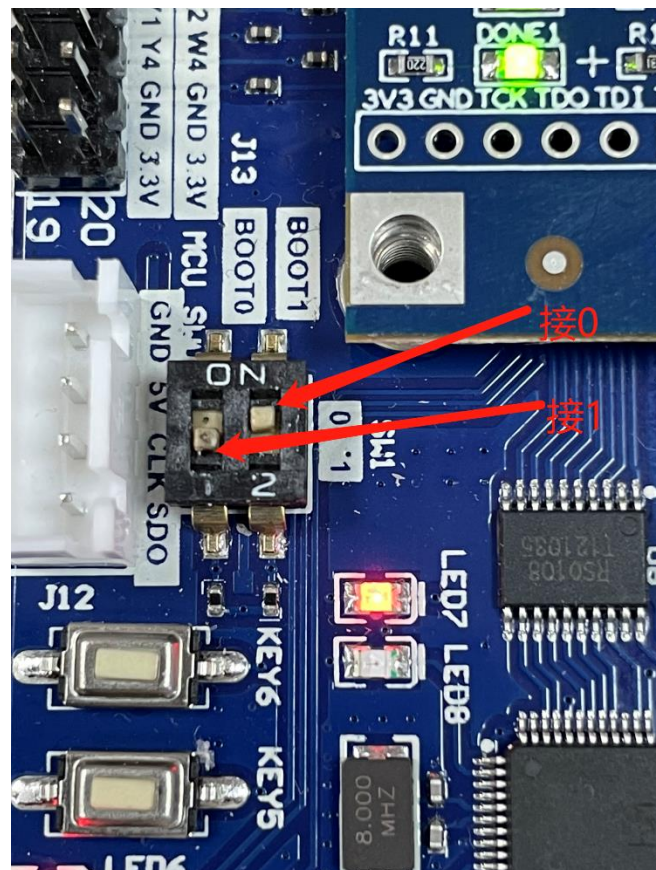




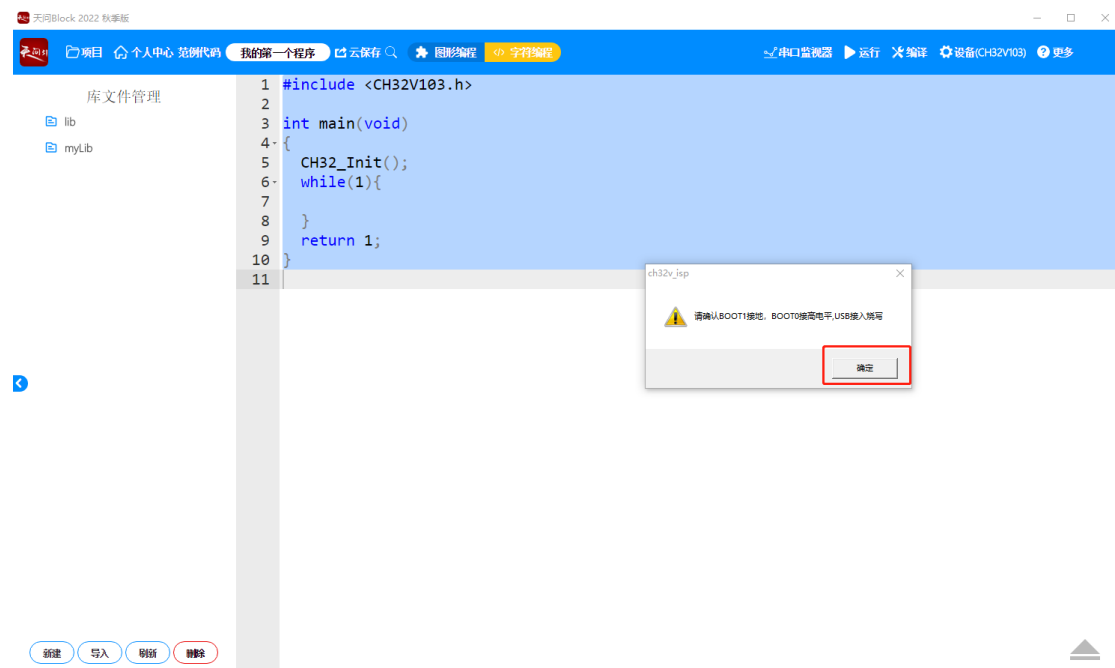
注：如果显示安装失败，点击卸载选项，进行重新安装。

2.2.2 烧写 U 盘固件设置

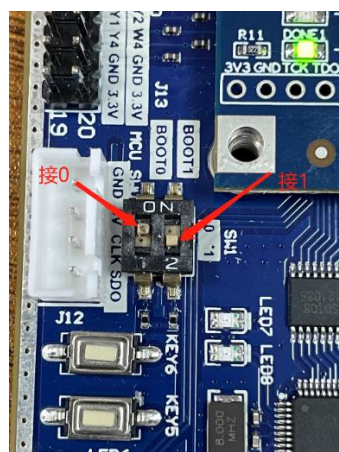
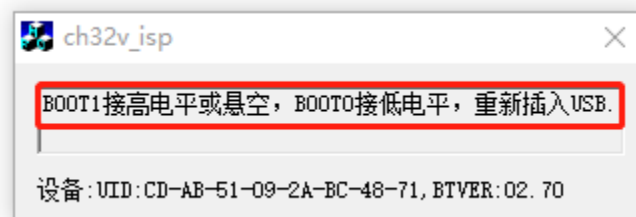
- 1) 在进行烧写U盘固件设置前，需将BOOT1接“0”，BOOT0接“1”，USB接入烧写口。



2) 点击更多，选择烧写U盘固件选项，会弹出如下界面，直接点击确定即可。



在此过程中会提示烧写成功后，会显示出如下BOOT设置界面。（请勿关闭ch32v_isp界面）。



断电后，拔出USB，然后将BOOT1接“1”，BOOT0接“0”后，再上电，将USB重新插入烧写口，电脑右下角将会自动提示有可打开的U盘。表示U盘固件设置成功。



关闭ch32v_isp界面。



若点击确定后, 提示没有找到设备, 先断电, 重新设置好后, 再上电, 进行上述步骤即可。

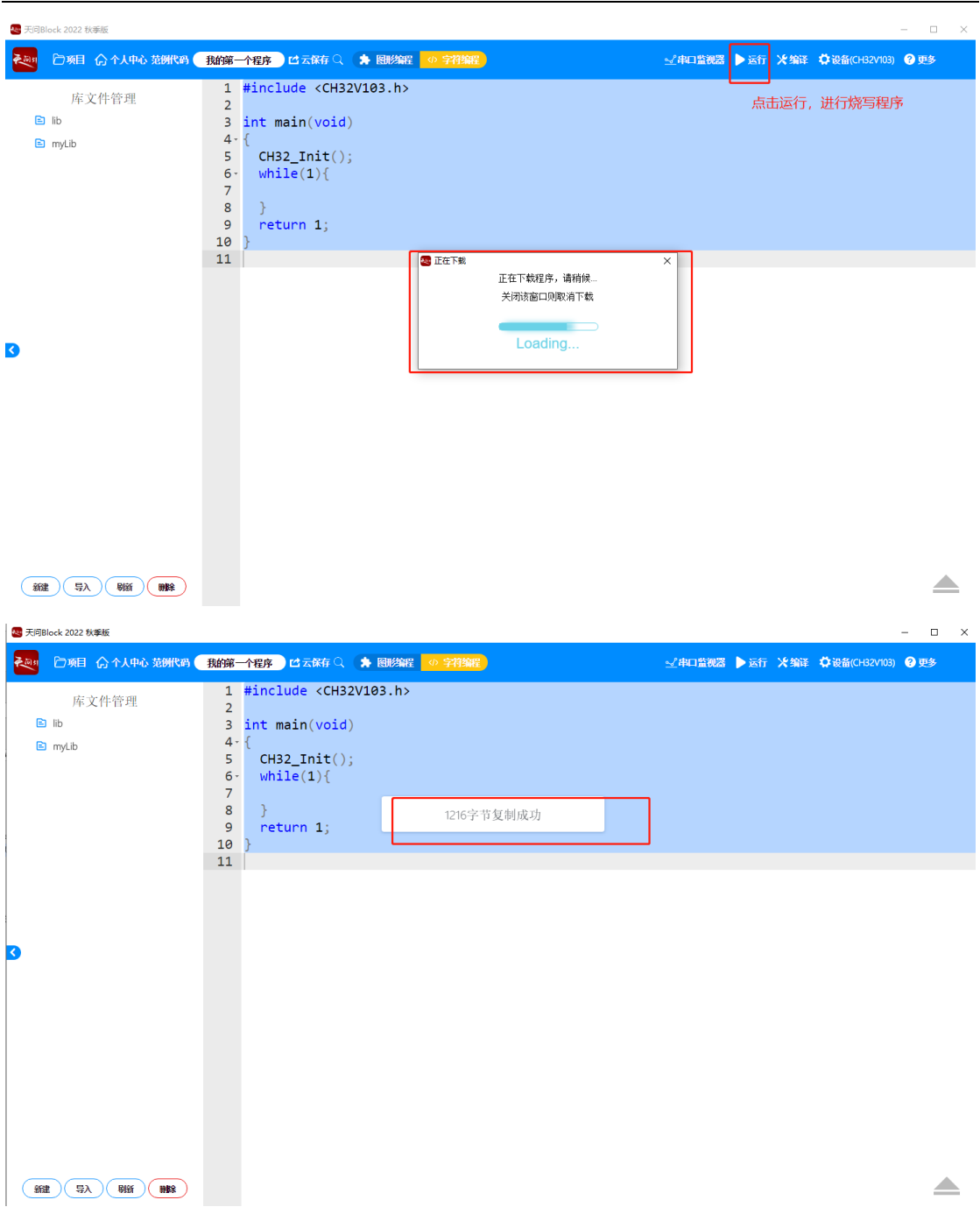


2.2.3 程序在线调试烧写-断电后, 程序会丢失

此烧写方法, 断电后, MCU中程序会丢失。

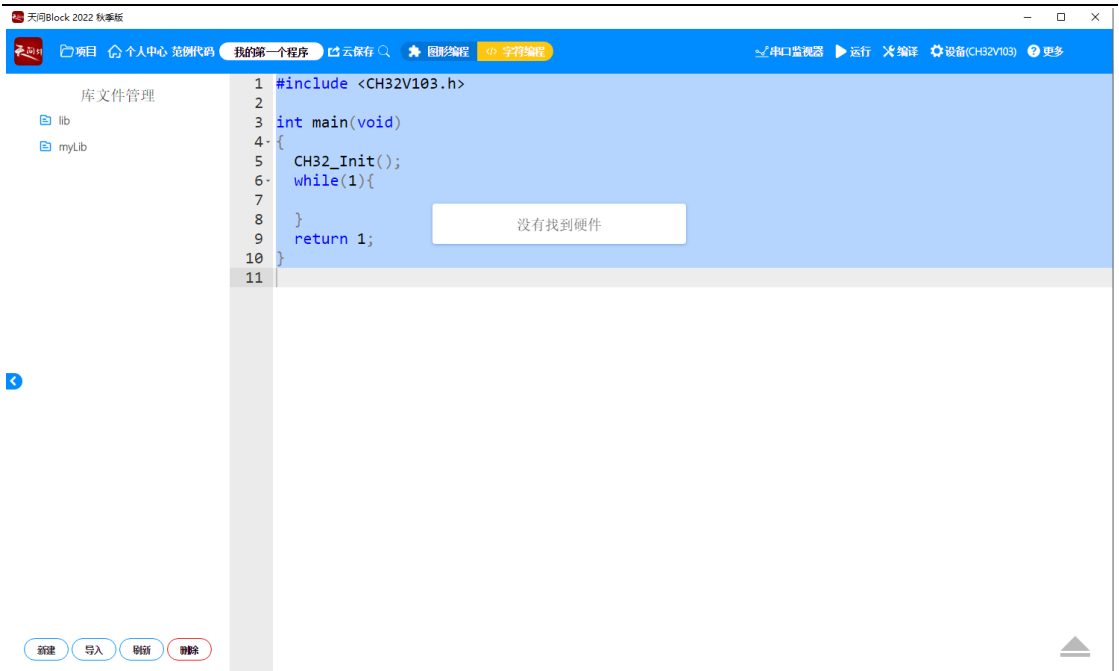
- 1) 点击运行按钮进行程序烧写。

中科亿海微电子科技有限公司(苏州)有限公司



烧写成功后，弹出****字节复制成功，表示程序烧写完成。

若烧写程序时，提示没有找到硬件，则进行复位（按键Key6）或重新断电再上电即可。



2.2.4 程序固化烧写-断电后，程序不会丢失

此烧写方法，断电后，MCU中程序不会丢失。

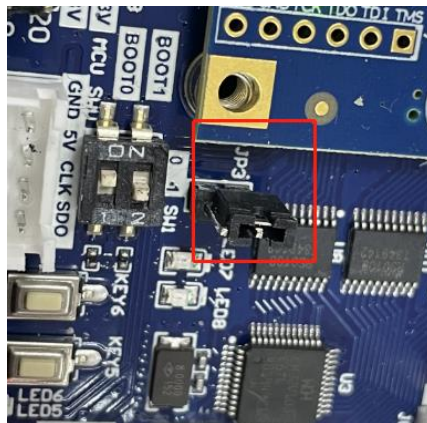
1) 点击运行按钮进行程序烧写。





烧写成功后，弹出****字节复制成功，表示程序烧写完成。

程序烧写完成后，需将跳线帽插入JP13上，如下图所示。



注：若要对程序进行再次烧写，需要将跳线帽拔出，复位（按键Key6）或重新断电再上电，进入U盘模式，即可进行程序烧写。

3. 实例

PA9对应管脚信息如下：

表3.1.2.1 PA9端口说明

MCU	FPGA	端口说明
PA9	AA21	PA9对应FPGA引脚为AA21

LED6 对应FPGA管脚为 N20。

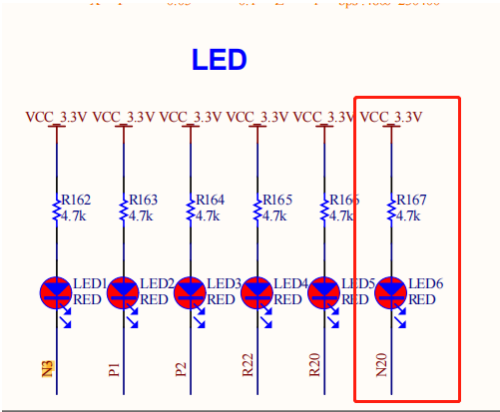


图3. 1. 2. 3 LED灯电路原理图

表3.1.2.2 信号管脚分配说明

信号	方向	FPGA管脚	端口说明
sys_clk	input	AB11	系统时钟50M
led_PA9_mcu	input	AA21	MCU输入FPGA的管脚
led_N20_fpga	output	N20	FPGA输出管脚

3.2.3 程序设计

3. 2. 3. 1 FPGA工程创建

1) 在菜单栏选项中， 点击File-Project-New创建新的工程。

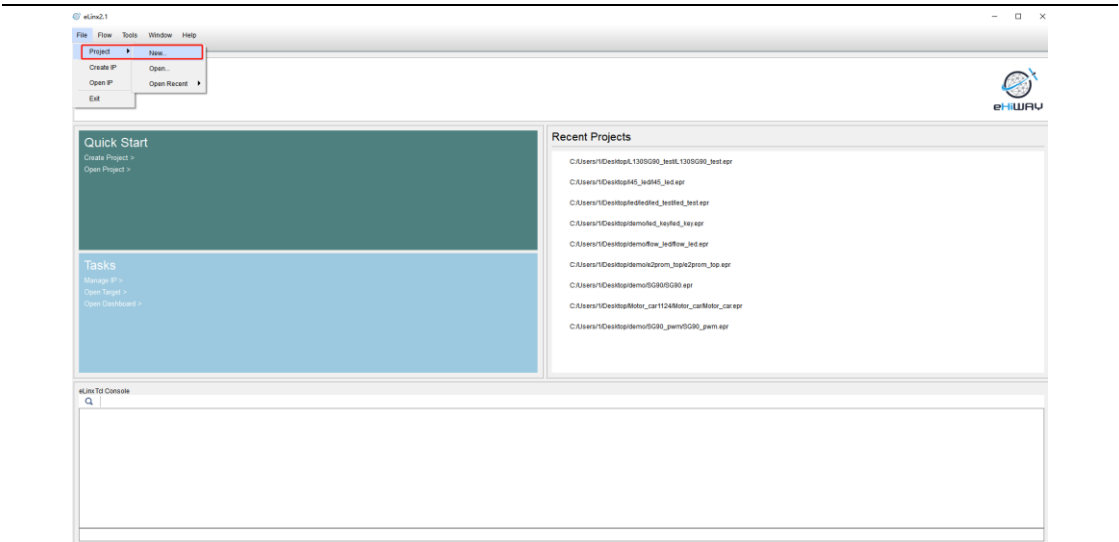


图3. 2. 3. 1 新建工程

2) 点击Next。

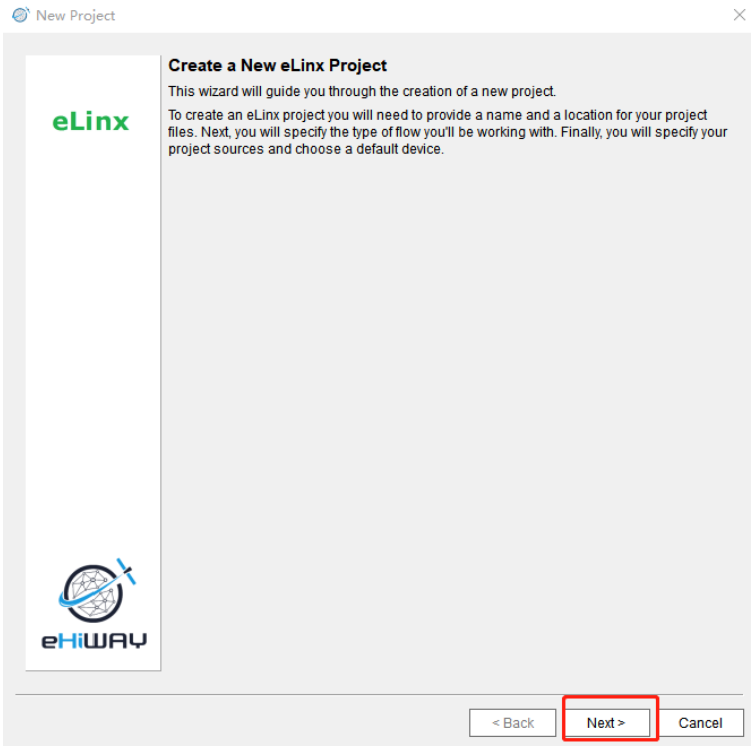


图3. 2. 3. 2 新建工程释义

3) 为新工程创建名称，该实例工程名为 LED，工程文件存放路径可自定义，再点击Next。

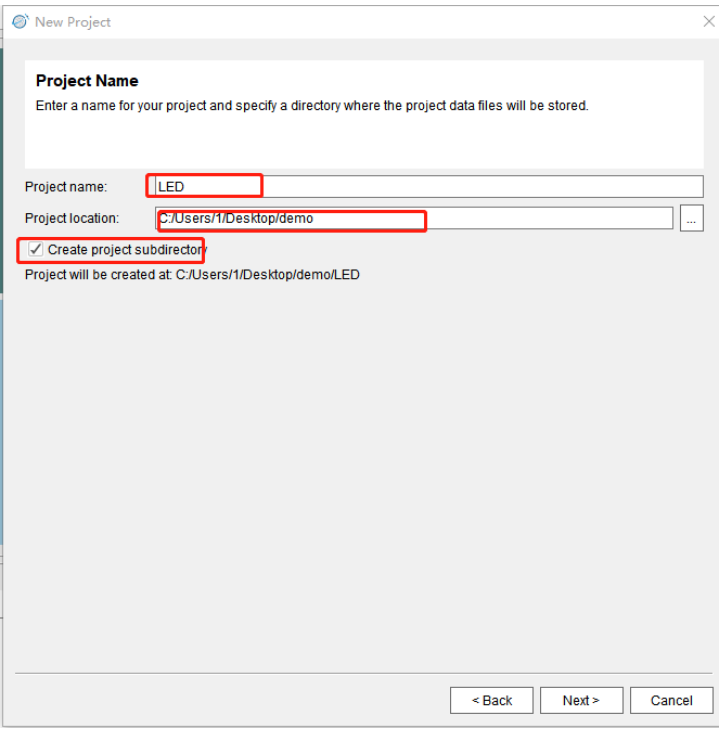


图3. 2. 3. 3 工程名的创建

4) 勾选 Do not specify sources at this time 选项，然后点击Next。

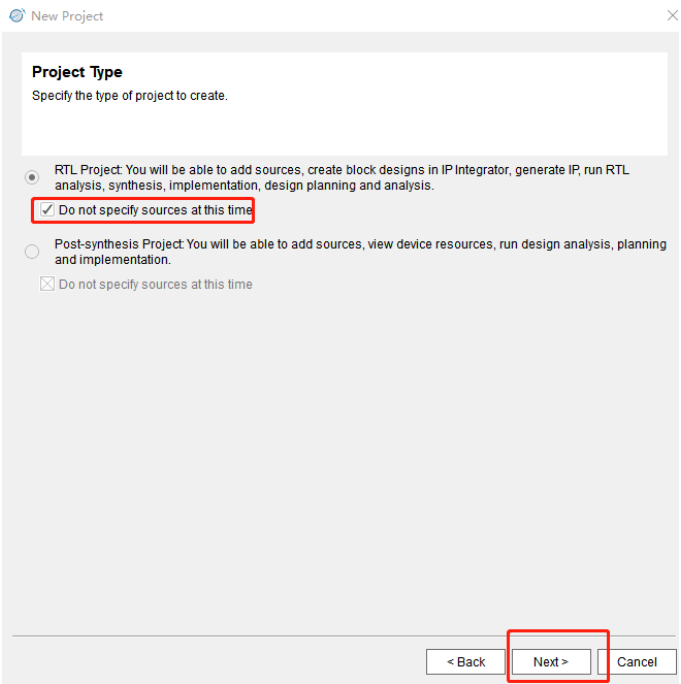


图3. 2. 3. 4 工程类型

5) 选择芯片型号，该工程项目选用芯片型号为EQ6HL130，封装为CSG484_H，然后点击Next。

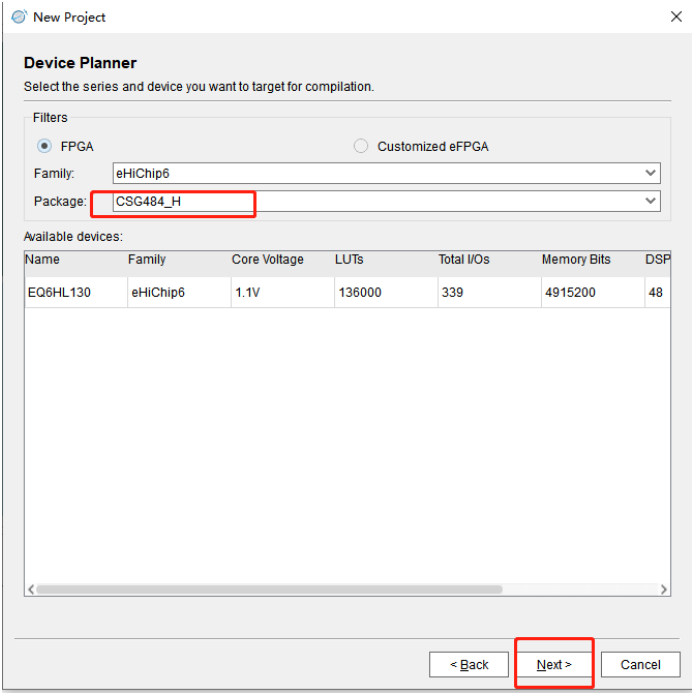


图3. 2. 3. 5 芯片器件选型

6) 点击Finish。

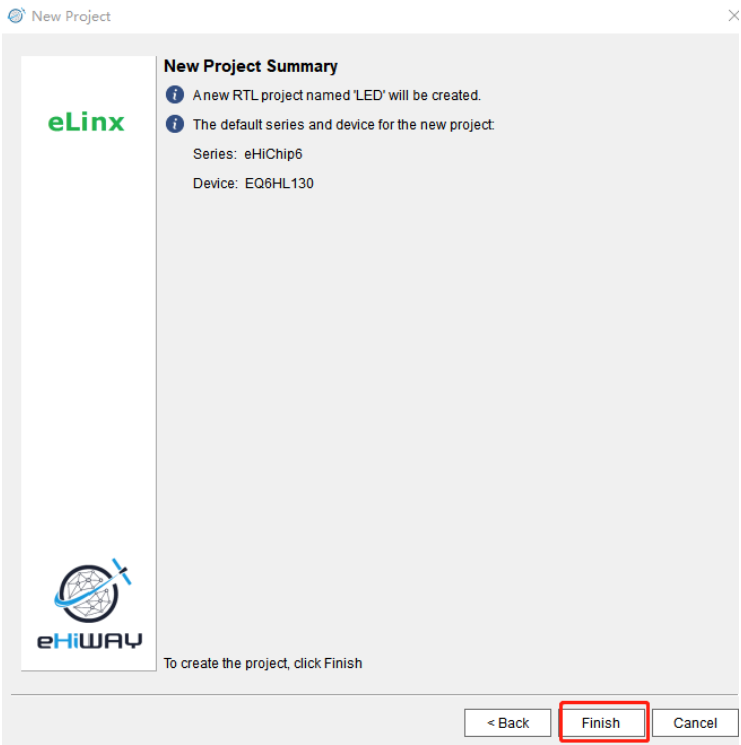


图3. 2. 3. 6 工程摘要

7) 为工程创建设计源文件，鼠标选择 Design Sources，点击鼠标右键，再选择

Add Sources选项，最后点击Next。

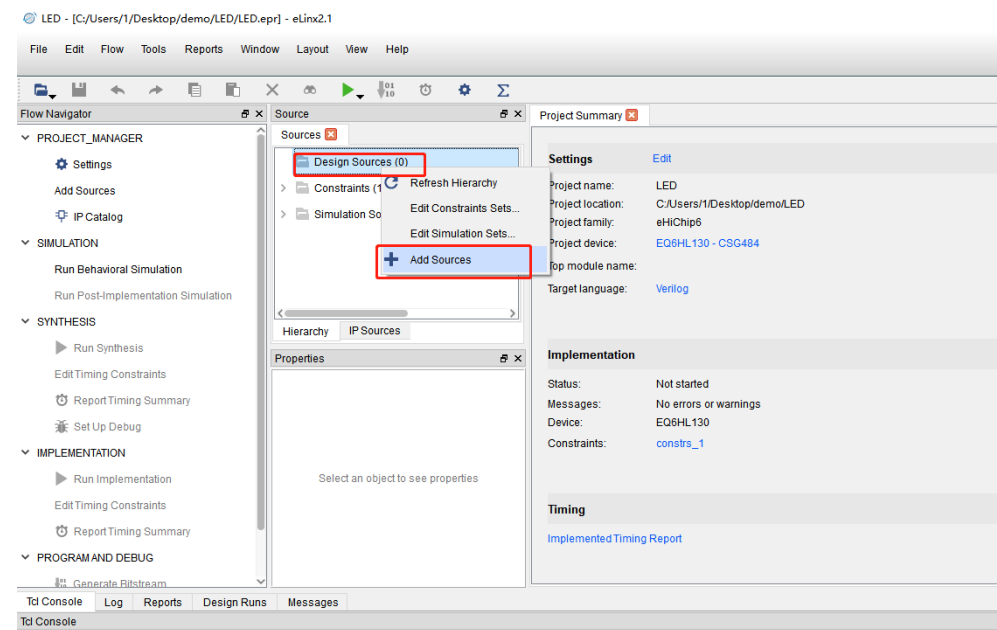


图3. 2. 3. 7 新建编译文件

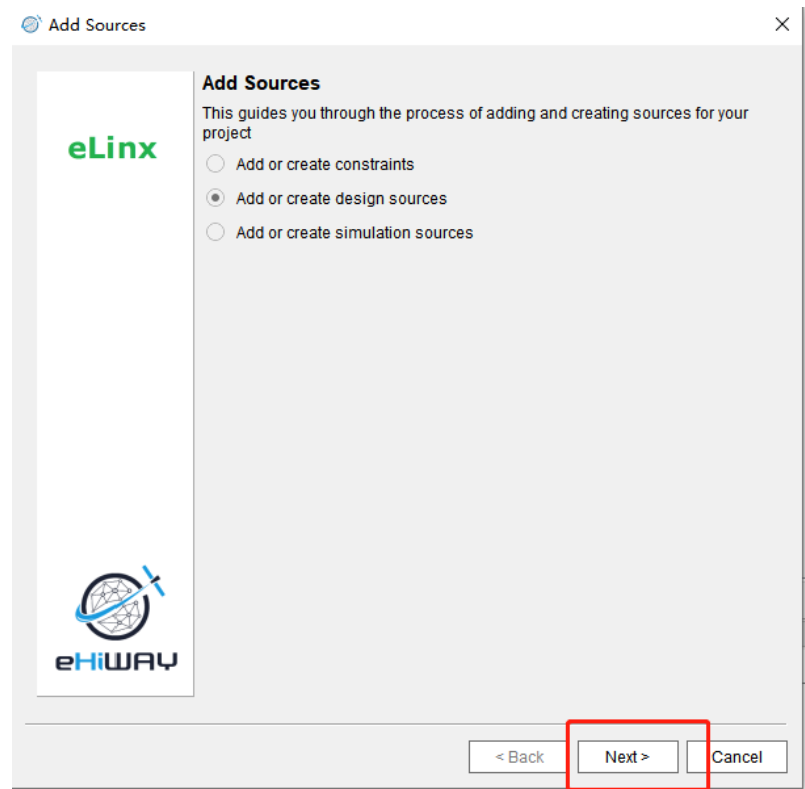


图3. 2. 3. 8 添加源文件

8) 点击Create File。

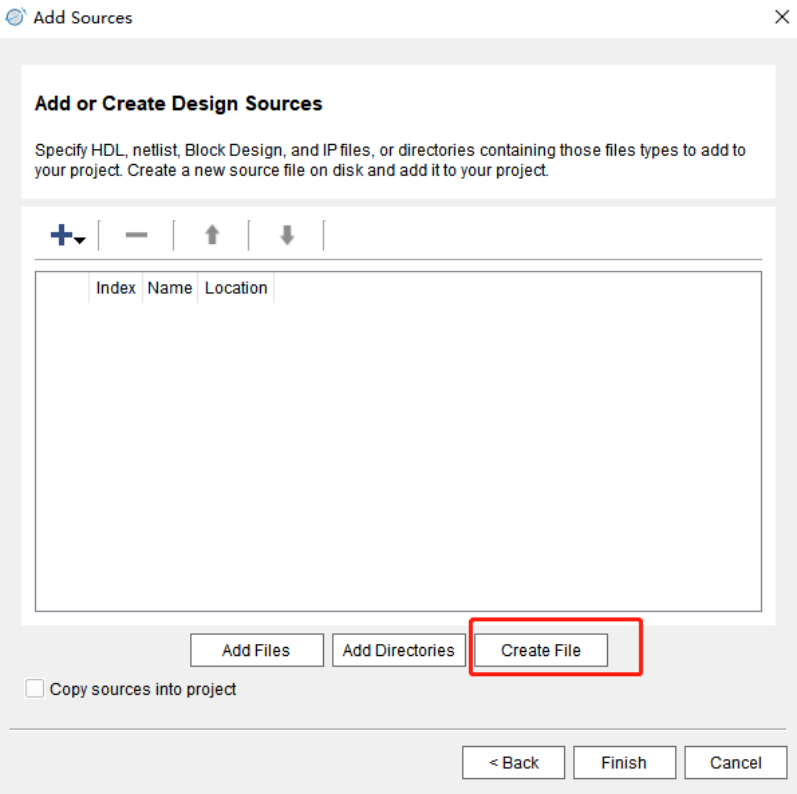


图3. 2. 3. 9 新建文件

9) 该文件名需和工程名保持一致，点击OK，再点击Finish。

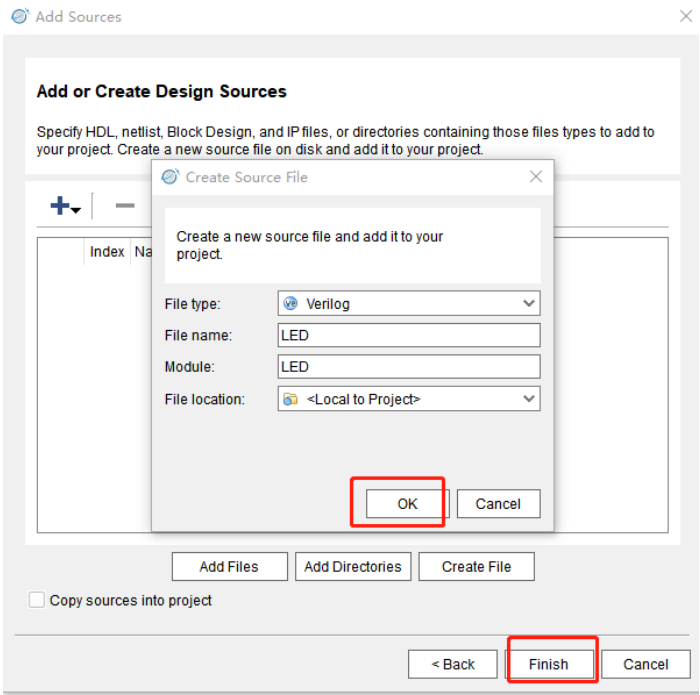


图3. 2. 3. 10 创建文件名

10) 双击Design Sources，在双击LED。

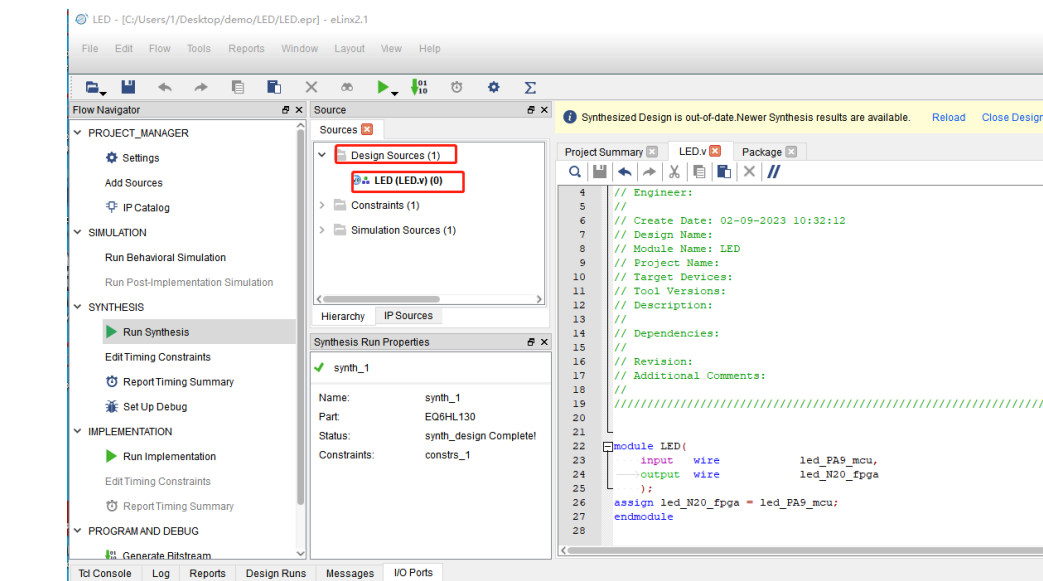


图3. 2. 3. 11 工程代码

11) 程序编写完成后，双击Run Synthesis，进行综合。

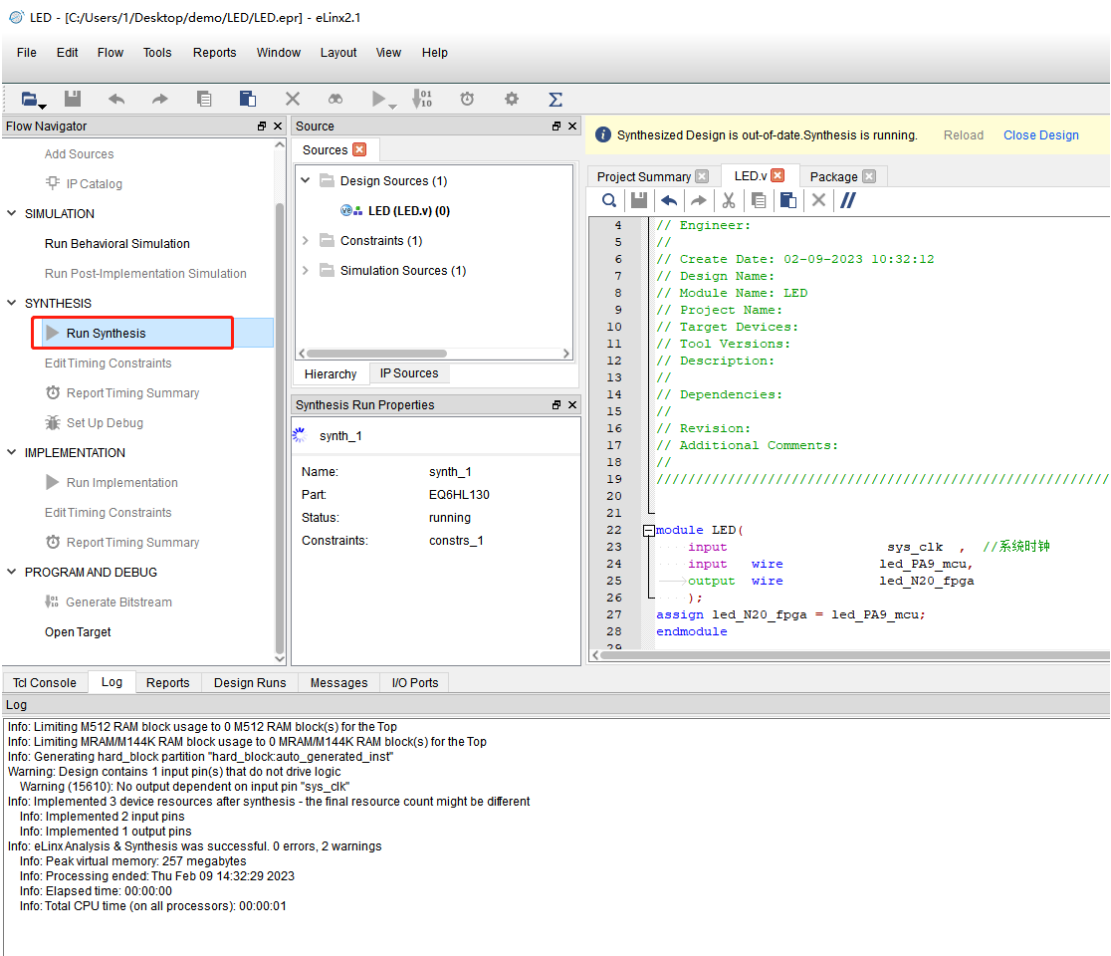


图3.2.3.12 工程综合

12) 综合完成之后开始绑定管脚，点击Tools/ IO Planning。

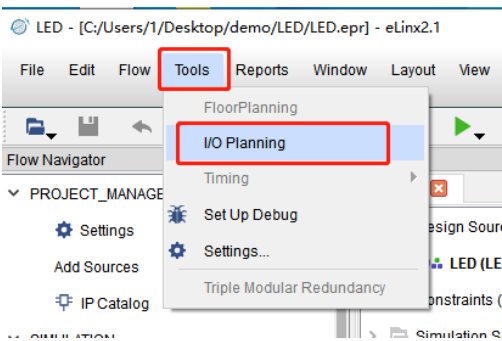


图3.2.3.13 管脚绑定

13) 这里下面可以进行管脚的绑定，同时可以清晰的看到管脚绑定的情况，点击保存。

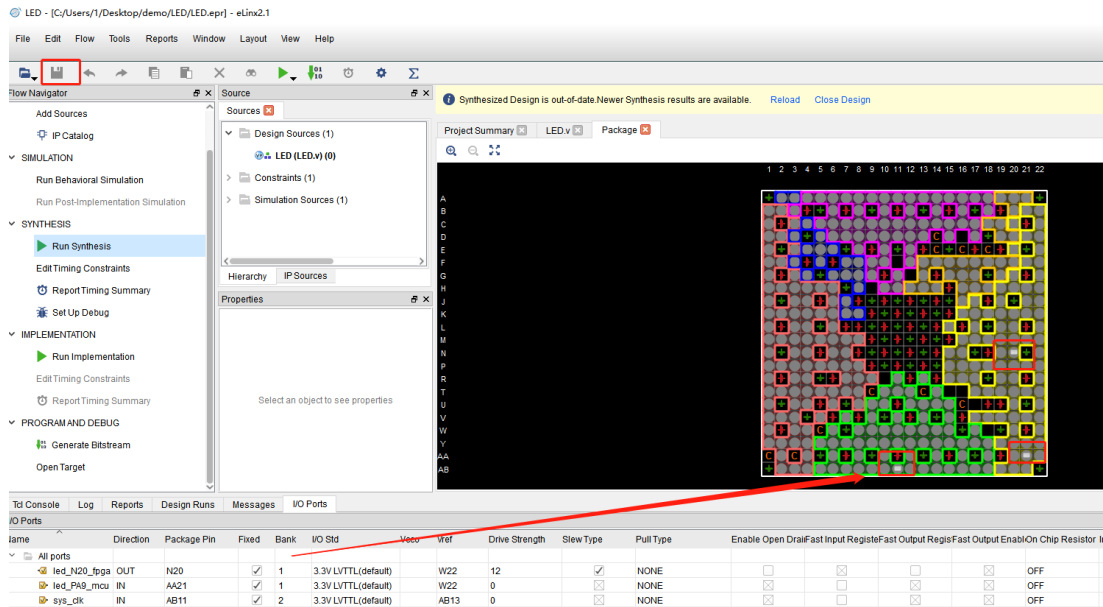


图3.2.3.14 管脚绑定示意图

14) 点击Run Implementation，进行布局布线。

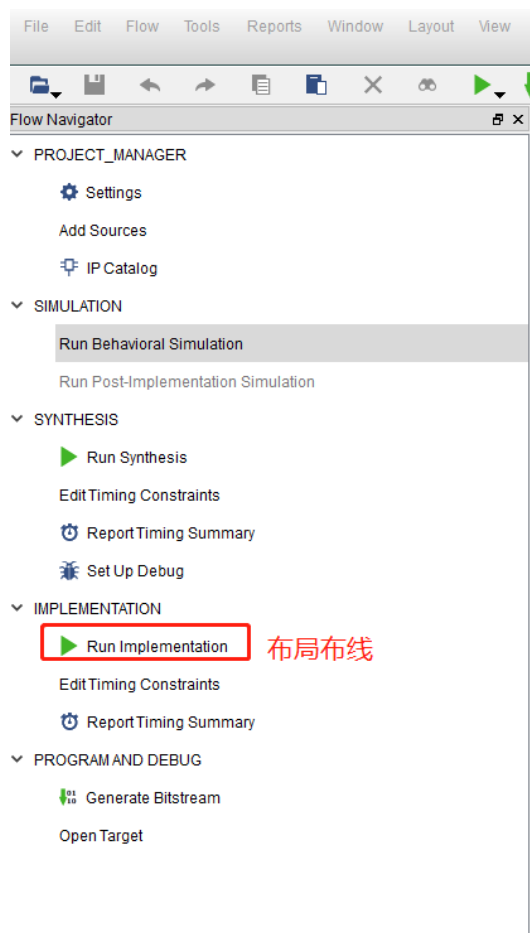


图3.2.3.15 布局布线

15) 没有报错即可生成bit流下载到芯片里，点击Generate Bitstream生成码流。

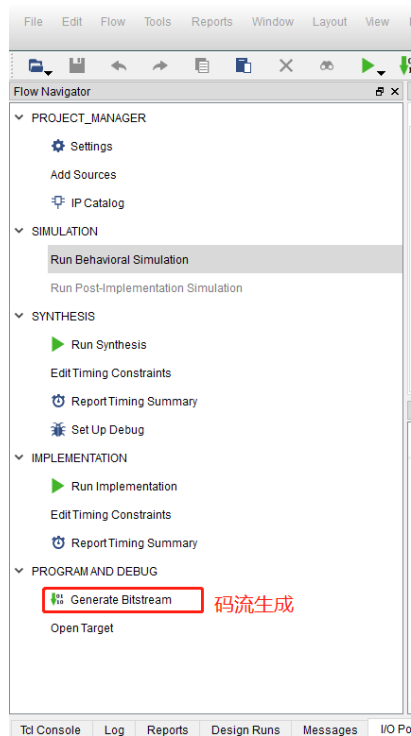


图3.2.3.16 码流生成

16) 码流生成之后，双击Open Target准备下载到芯片里。

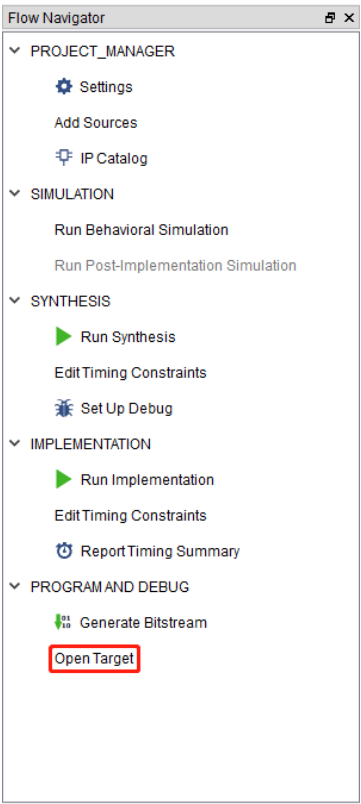


图3.2.3.17 程序下载

17) 这里选择FLASH下载模式，然后点击Auto Detect，再点击Start，等待程序烧录成功即可。

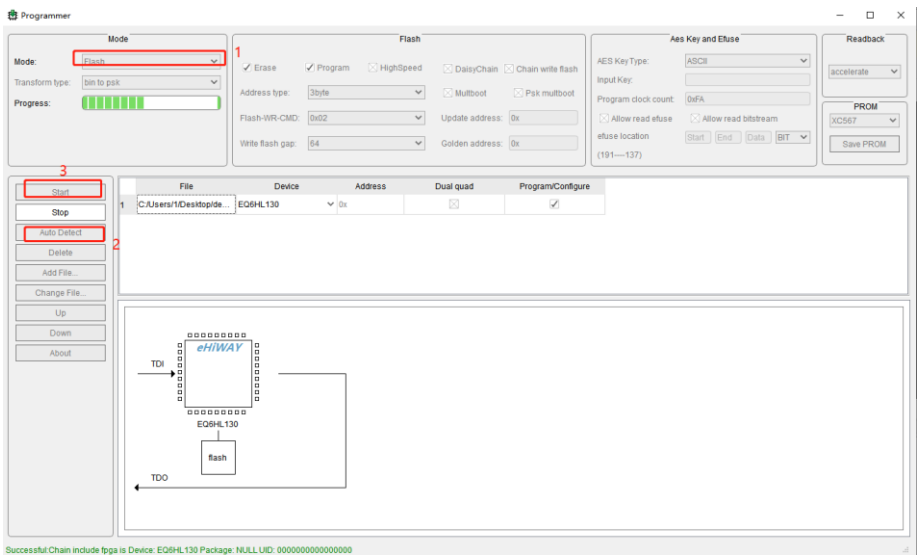


图3.2.3.18 程序下载界面

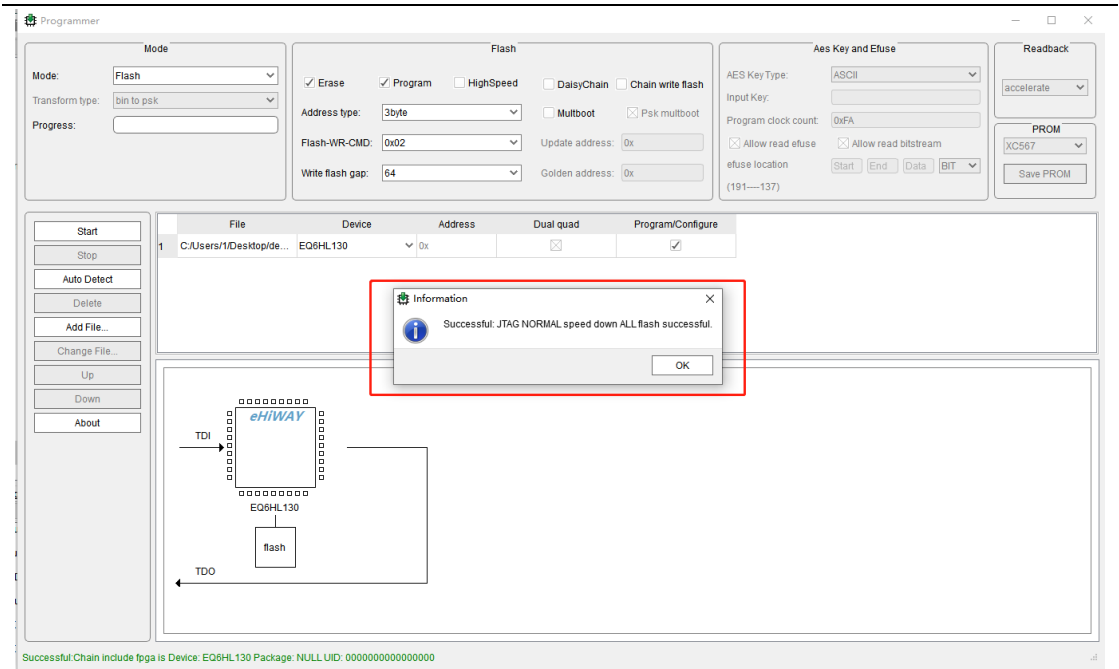


图3. 2. 3. 19 下载成功界面

3. 2. 3. 2 MCU工程创建

1) 这双击打开天问Block软件。



图3. 2. 3. 1 天问软件

2) 点击设备按钮，MCU芯片选择CH32V103。

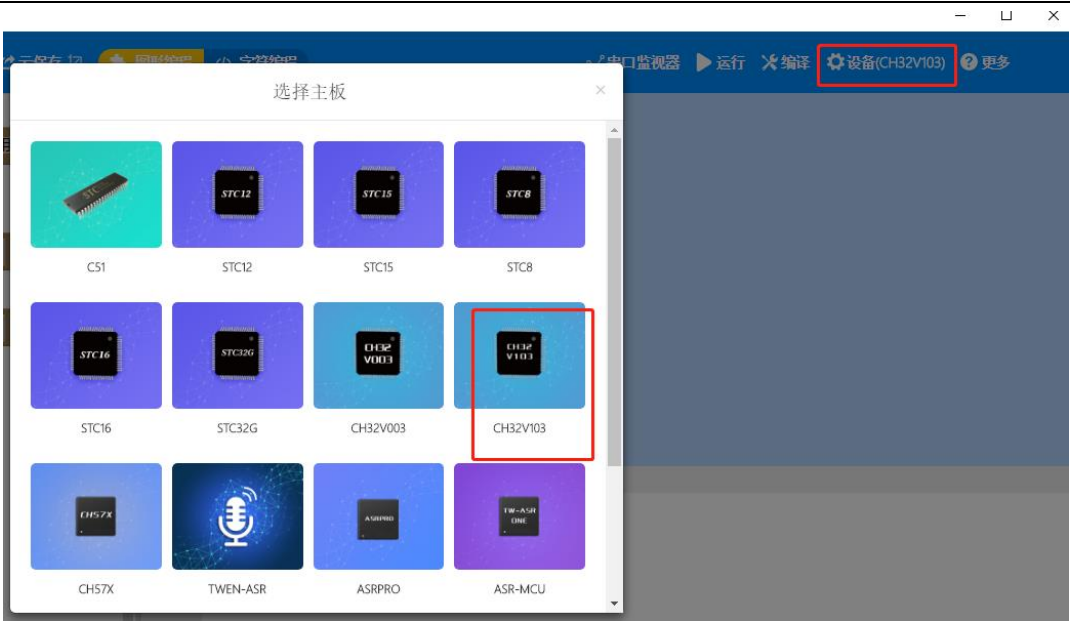


图3. 2. 3. 2 芯片选型

3) 点击范例代码，选择GPIO控制LED，再选择板载灯闪烁一秒亮一秒灭。

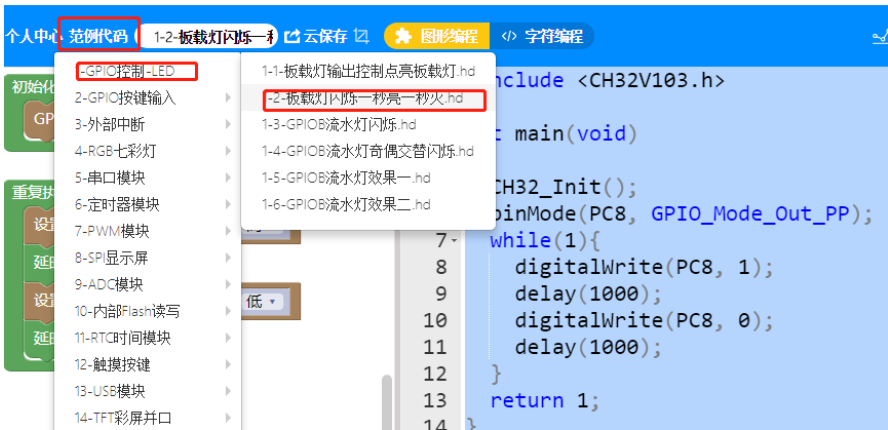


图3. 2. 3. 3 范例代码

4) 将设置引脚改为PA9。

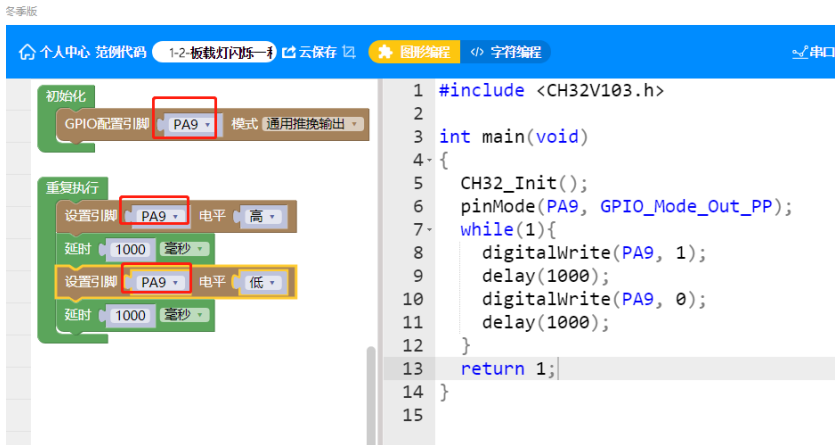


图3. 2. 3. 3 管脚选择

5) 点击编译按钮。

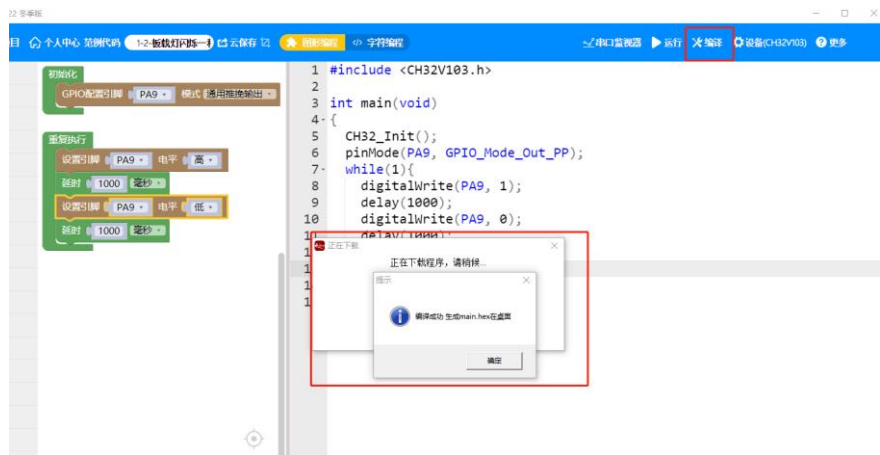


图3. 2. 3. 4 编译

6) 点击运行

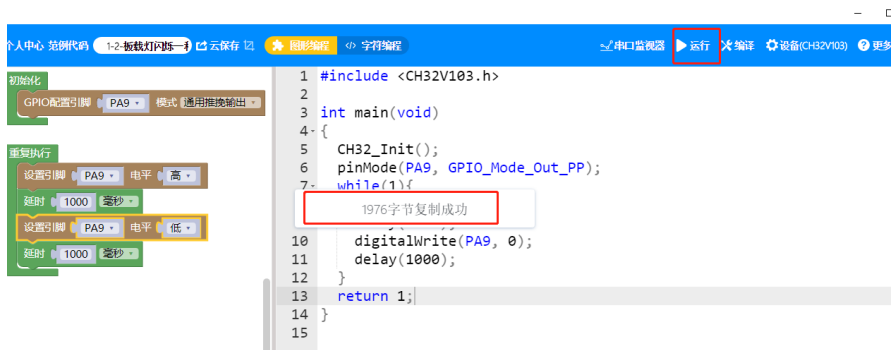


图3. 2. 3. 5 程序下载

3.2.4 实验结果

运行结果如下所示:

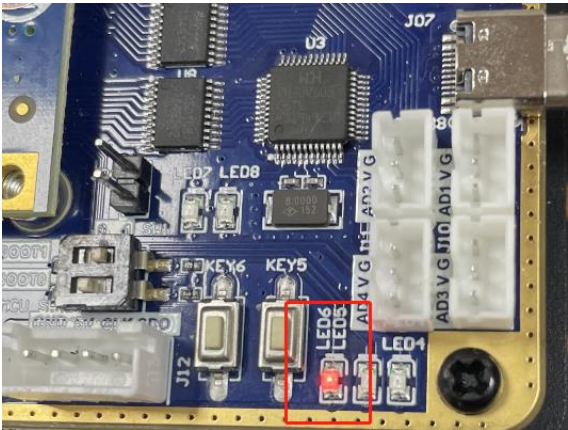


图3. 1. 4. 6 上板验证结果

3.2实例—舵机

3.2.1 实验简介

舵机的控制一般需要一个20ms左右的时基脉冲，该脉冲的高电平部分一般为0.5ms~2.5ms范围内的角度控制脉冲部分。以180度角度伺服为例，那么对应的控制关系是这样的：

表3.2.1.1 舵机控制

脉冲宽度	旋转角度
0.5ms	0度
1.0ms	45度
1.5ms	90度
2.0ms	135度
2.5ms	180度

通过设定舵机的占空比，改变舵机的旋转角度。

3.2.2 硬件设计

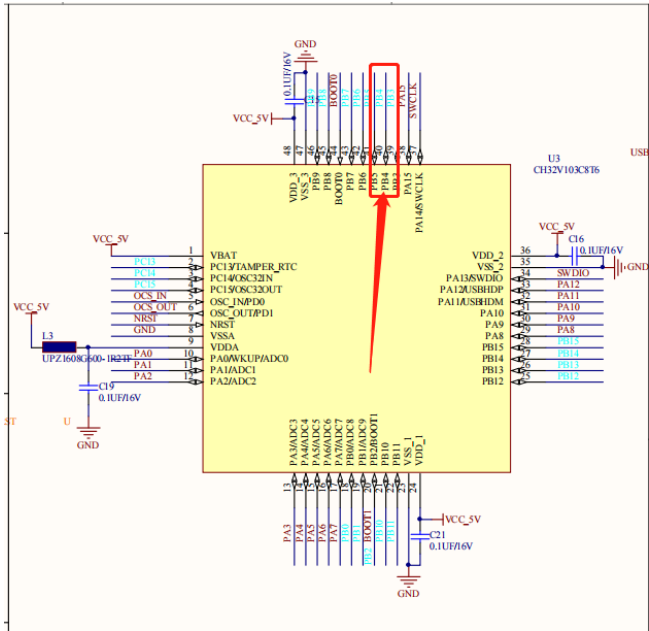


图3.2.2.1 MCU引脚

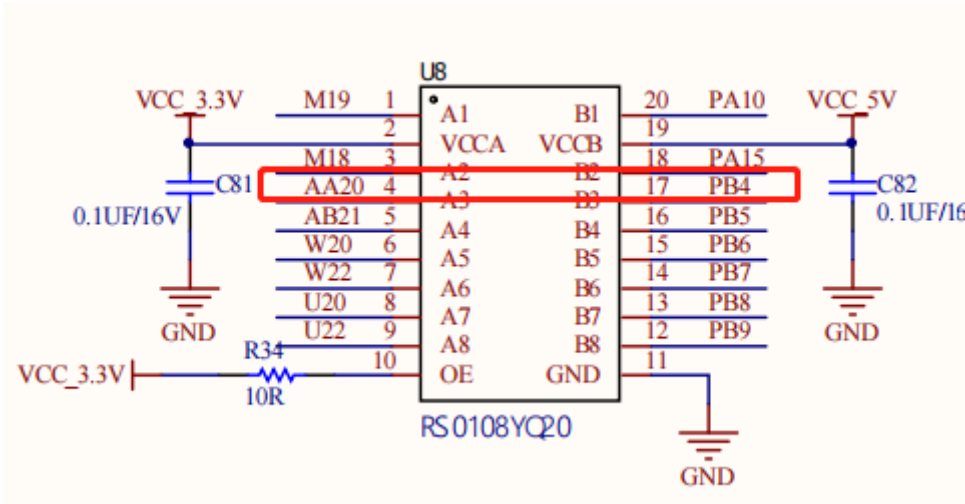


图3.2.2.2 PB4对应FPGA引脚

PB4对应管脚信息如下：

表3.2.2.1 PA9端口说明

MCU	FPGA	端口说明
PB4	AA20	PB4对应FPGA引脚为AA20

舵机对应FPGA管脚为 N20。

表3.2.2.2 信号管脚分配说明

信号	方向	FPGA管脚	端口说明
sys_clk	input	AB11	系统时钟50M
i_servo_PB4	input	AA20	MCU输入FPGA的管脚
o_servo_L4	output	L4	FPGA输出管脚

3.2.3 程序设计

```
#define PWM_DUTY_MAX 1000//PWM最大占空比值

#include <CH32V103.h>
#include "CH32V_PWM.h"
```

图3.2.3.1 PWM最大占空比

PWM_DUTY_MAX 表示舵机PWM的最大占空比值。

```
int main(void)
{
    CH32_Init();
    PWM_Init(TIM3_CH1, PB4, 50, 0);
    while(1){
        PWM_Duty_Update(TIM3_CH1, (0 / 1.8 + 25));
        delay(1000);
        PWM_Duty_Update(TIM3_CH1, (45 / 1.8 + 25));
        delay(1000);
        PWM_Duty_Update(TIM3_CH1, (90 / 1.8 + 25));
        delay(1000);
        PWM_Duty_Update(TIM3_CH1, (135 / 1.8 + 25));
        delay(1000);
        PWM_Duty_Update(TIM3_CH1, (180 / 1.8 + 25));
        delay(1000);
    }
    return 1;
}
```

图3.2.3.2 主程序

在CH32V_PWM.h文件中，可以看到PWM初始化，各参数的含义，第一个参数表示管脚的PWM定时器和定时器通道，第二是参数表示管脚，第三个参数表示频率，第四个参数表示占空比，通过数据手册可查询到，重映射功能中对应TIM3_CH1。

38	38	50	PA15	I/O	PA15		TIM2_CH1/TIM2_ETR /SPI1_NSS
-	-	51	PC10	I/O	PC10		UASRT3_TX
-	-	52	PC11	I/O	PC11		USART3_RX
-	-	53	PC12	I/O	PC12		USART3_CK
-	-	54	PD2	I/O	PD2	TIM3_ETR	
39	39	55	PB3	I/O	PB3		TRACESWO/TIM2_CH2 /SPI1_SCK
40	40	56	PB4	I/O	PB4		TIM3_CH1/SPI1_MISO
41	41	57	PB5	I/O	PB5	I2C1_SMBAI	TIM3_CH2/SPI1_MOSI
42	42	58	PB6	I/O/A	PB6	I2C1_SCL/TIM4_CH1	USART1_TX
43	43	59	PB7	I/O/A	PB7	I2C1_SDA/TIM4_CH2	USART1_RX
44	44	60	BOOT0	I	BOOT0		
45	45	61	PB8	I/O/A	PB8	TIM4_CH3	I2C1_SCL
46	46	62	PB9	I/O/A	PB9	TIM4_CH4	I2C1_SDA
47	47	63	Vss,3	P	Vss,3		
48	48	64	Vss,3	P	Vss,3		

图3.2.3.3 芯片数据手册

FPGA程序中，将单片机输出的信号PB4作为，FPGA输入的信号，具体程序如下图所示。


```
module Servo(  
    →input          sys_clk,  
    →input          i_servo_PB4,  
    →output         o_servo_L4  
    ...  
).  
    assign o_servo_L4 = i_servo_PB4;  
endmodule
```

图3.2.3.4 FPGA直连

3.2.4 实验结果

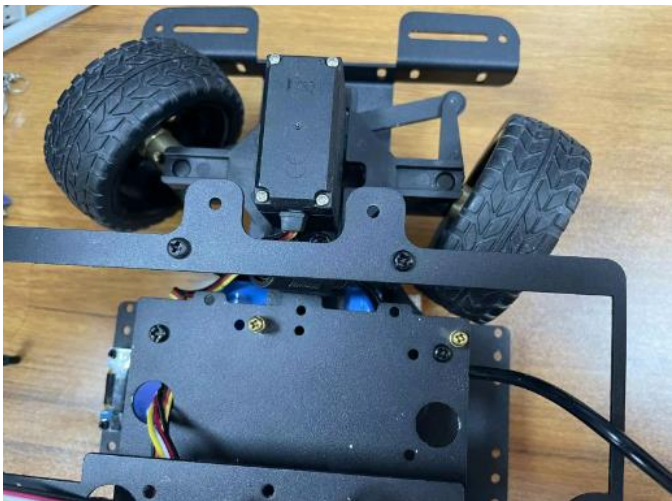


图3.2.4.1 舵机旋转角度结果

3.3实例一电机

3.3.1 实验简介

所谓PWM，就是脉冲宽度调制技术，其具有两个很重要的参数：频率和占空比。频率，就是周期的倒数；占空比，就是高电平在一个周期内所占的比例。

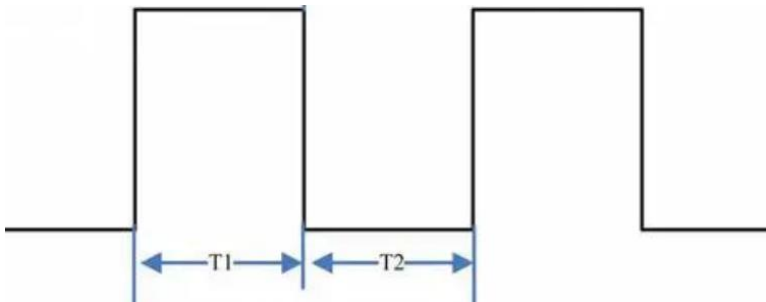


图3.3.1.1 PWM占空比

在上图中，频率F的值为 $1/(T1+T2)$ ，占空比D的值为 $T1/(T1+T2)$ 。通过改变

图3.3.2.1 MCU引脚

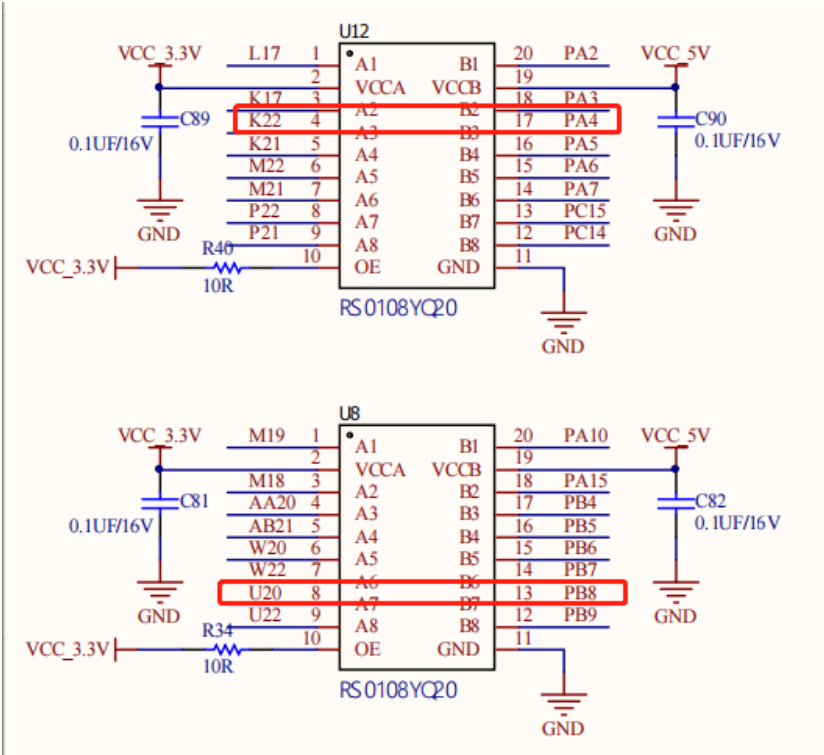


图3.3.2.2 电机对应FPGA引脚

电机对应管脚信息如下：

表3.3.2.1 电机端口说明

MCU	FPGA	端口说明
PB8	U20	PB8对应FPGA引脚为U20
PA4	K22	PA4对应FPGA引脚为K22

表3.3.2.2 信号管脚分配说明

信号	方向	FPGA管脚	端口说明
sys_clk	input	AB11	系统时钟50M
i_PWM_PB8	input	U20	FPGA输入管脚
i_PWM_PA4	input	K22	FPGA输入管脚
o_motor_IN1A	output	L20	FPGA输出管脚
o_motor_IN1B	output	M20	FPGA输出管脚

3.3.3 程序设计

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE); //使能TIM4时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE); //使能GPIOC外设
```

图3.3.3.1 开启时钟

我们操作的外围设备一般都是位于AHB和APB总线上，而AHB可以引伸出AHB1、AHB2，甚至AHB3。同样APB也存在APB1、APB2等，而操作外设是通过外设总线来实现，只有外设总线有时钟了才能操作外设，所以在对外设设备操作时，需要开启时钟。

```
GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //设置为复用推挽输出
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8; //配置PB8引脚
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //设置输出速度：50MHz
GPIO_Init(GPIOB, &GPIO_InitStructure); //GPIO初始化
```

图3.3.3.2 配置GPIO引脚

GPIO口主要有两大功能，输出和输入。因为在STM32中有很多端口重映射，GPIO口也经常重映射成各外设的输出口，因此输出中又分为两大类：通用输出和复用功能映射；这两种里面都包括两种模式：推挽输出和开漏输出。输入功能则包括了四种模式：模拟输入、浮空输入、下拉输入、上拉输入。

```
TIM_InternalClockConfig(TIM4);

TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1; //时钟分频因子
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up; //TIM计数模式，向上计数模式
TIM_TimeBaseInitStructure.TIM_Period = 100-1; //指定下次更新事件时要加载到活动自动重新加载寄存器中的周期值。
TIM_TimeBaseInitStructure.TIM_Prescaler = 0; //指定用于划分TIM时钟的预分频器值。
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0; //重复计数，就是重复溢出多少次才给来一次溢出中断，
TIM_TimeBaseInit(TIM4, &TIM_TimeBaseInitStructure); //根据指定的参数初始化TIMx的时间基数单位

TIM_OCInitTypeDef TIM_OCInitStructure;
TIM_OCStructInit(&TIM_OCInitStructure);
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1; //指定TIM模式
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High; //指定输出极性。
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //指定TIM输出比较状态，即使能比较输出
TIM_OCInitStructure.TIM_Pulse = 0; //指定要加载到捕获比较寄存器中的脉冲值。
TIM_OC1Init(TIM4, &TIM_OCInitStructure);
TIM_OC2Init(TIM4, &TIM_OCInitStructure);
TIM_OC3Init(TIM4, &TIM_OCInitStructure);
TIM_OC4Init(TIM4, &TIM_OCInitStructure);

TIM_Cmd(TIM4, ENABLE);
```

图3.3.3.2 定时器配置

定时器的配置主要配置4个部分：自动重装载寄存器数值、预分频系数、时钟分割（一般的定时功能用不到，可设为0）、计数模式。在本例程中，自动重装载寄存器数值为100，预分频系数为0+1，时钟分割为0，计数模式为向上技术模式。

```
void PWM_SetCompare3(uint16_t Compare)
{
    TIM_SetCompare3(TIM4, Compare);
}
```

图3.3.3.3 定时器配置

在CH32V103芯片数据手册中，PB8默认复用功能：TIM4_CH3，进行定时器设定时，需要用到定时器4通道3，TIM_SetCompare3中Compare是输出高电平的时间，可通过修改Compare值，间接输出不同占空比的PWM。

```
void Motor_SetSpeed(int8_t Speed)
{
    if (Speed >= 0)
    {
        GPIO_SetBits(GPIOA, GPIO_Pin_4);
        PWM_SetCompare3(Speed);
    }
    else
    {
        GPIO_ResetBits(GPIOA, GPIO_Pin_4);
        PWM_SetCompare3(-Speed);
    }
}
```

图3.3.3.4 电机正反转

通过向 Motor_SetSpeed函数中，传递数值的大小和正负，进而控制电机的正反转，以达到控制电机的目的。

```
module Motor(
    input sys_clk,
    input i_PWM_PB8,
    input i_PWM_PA4,
    output o_motor_IN1A,
    output o_motor_IN1B,
    ...);
    assign o_motor_IN1A=i_PWM_PB8;
    assign o_motor_IN1B=i_PWM_PA4;
endmodule
```

图3.3.3.5 电机FPGA直连

单片机输出的PB8、PA4，作为FPGA的输入，将信号分别传递给电机的IN1A、IN1B引脚，即电机的动力线1（引脚1）和动力线2（引脚6）。

3.3.4 实验结果



图3.3.4.1 电机

3.4实例—OLED

3.4.1 实验简介

本文采用的是4针的0.96寸OLED显示进行讲解，采用的是I2C协议，驱动芯片为SSD1306，SSD1306的每页包含了128个字节，总共8页，一共是 128*64 的点阵大小，屏幕整体分辨率为128*64，有黄蓝、白、蓝三种颜色可选，供电范围为3~5.5V，使用OLED屏进行数字显示。



图3.4.1.1 OLED显示屏

下面是引脚功能介绍。

3.4.2 硬件设计



OLED对应管脚信息如下：

表3.3.2.1 电机端口说明

MCU	FPGA	端口说明
PB6	W20	PB8对应FPGA引脚为U20
PB7	W22	PA4对应FPGA引脚为K22

表3.3.2.2 信号管脚分配说明

信号	方向	FPGA管脚	端口说明
sys_clk	input	AB11	系统时钟50M
i_SCL_PB6	input	W20	FPGA输入管脚
i_SDA_PB7	input	W22	FPGA输入管脚
o_SCL1	output	K1	FPGA输出管脚
o_SDA1	output	K2	FPGA输出管脚

3.4.3 程序设计

```
/*引脚配置*/
#define OLED_W_SCL(x)  GPIO_WriteBit(GPIOB, GPIO_Pin_6, (BitAction)(x))
#define OLED_W_SDA(x)  GPIO_WriteBit(GPIOB, GPIO_Pin_7, (BitAction)(x))
```

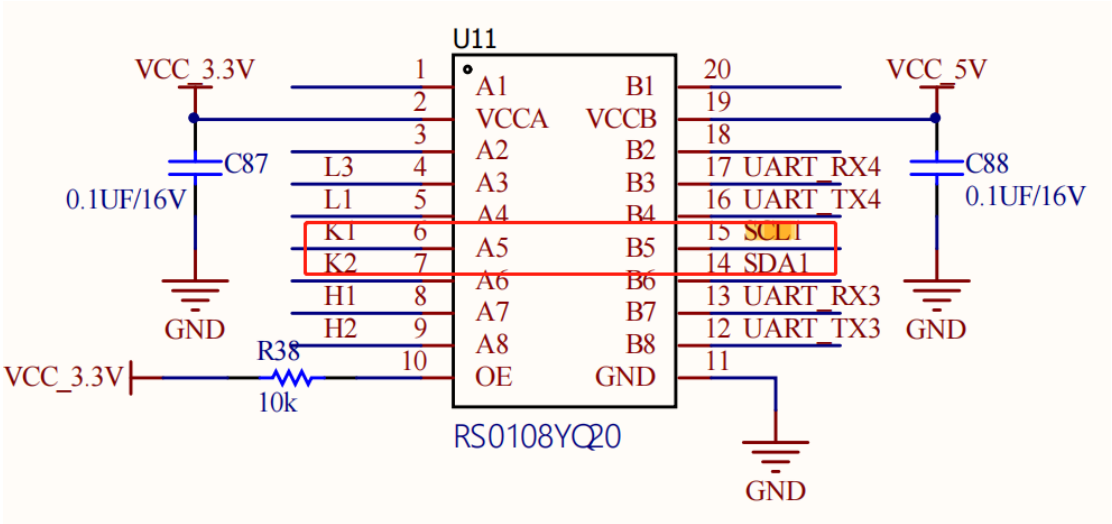


图3.4.3.1 OLED引脚

OLED显示屏引脚分配定义，根据图3.4.2.1和图3.4.3.1可以看出，PB6引脚与OLED显示屏的SCL引脚相连，PB7引脚与OLED显示屏的SDA引脚相连。


```
/*引脚初始化*/
void OLED_I2C_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_OD;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_Init(GPIOB, &GPIO_InitStructure);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    OLED_W_SCL(1);
    OLED_W_SDA(1);
}
```

图3.4.3.2 引脚初始化

IIC（Inter-Integrated Circuit）是一个多主从的串行总线，又叫I2C，是由飞利浦公司发明的通讯总线，属于半双工同步传输类型总线。IIC总线是非常常见的数据总线，仅仅使用两条线就能完成多机通讯，一条SCL时钟线，另外一条双向数据线SDA。

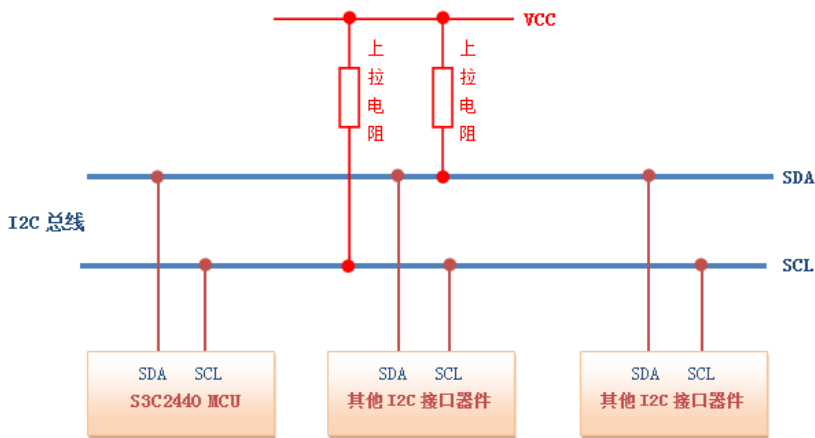


图3.4.3.3 IIC总线

表3.4.3.1 信号管脚分配说明

单片机	OLED屏
VCC	VCC
GND	GND
IIC_SDA(PB7)	SDA
IIC_SCL(PB6)	SCL

SDA线上的数据在SCL时钟“高”期间必须是稳定的，只有当SCL线上的时钟信号为低时，数据线上的“高”或“低”状态才可以改变。输出到SDA线上的每个字节必须是8位，数据传送时，先传送最高位（MSB），每一个被传送的字节后面都必须跟随一位应答位（即一帧共有9位）。

当一个字节按数据位从高位到低位的顺序传输完后，紧接着从设备将拉低SDA线，回传给主设备一个应答位ACK，此时才认为一个字节真正的被传输完成，如果一段时间内没有收到从机的应答信号，则自动认为从机已正确接收到数据。

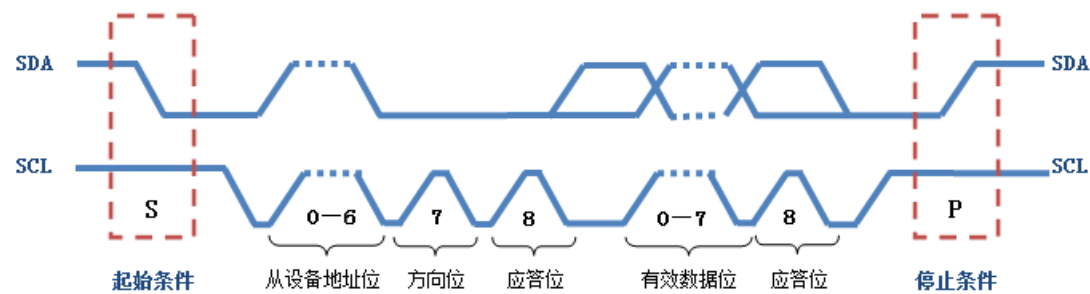


图3.4.3.4 IIC传输示意图

```
/**
 * @brief I2C开始
 * @param 无
 * @retval 无
 */
void OLED_I2C_Start(void)
{
    OLED_W_SDA(1);
    OLED_W_SCL(1);
    OLED_W_SDA(0);
    OLED_W_SCL(0);
}

/**
 * @brief I2C停止
 * @param 无
 * @retval 无
 */
void OLED_I2C_Stop(void)
{
    OLED_W_SDA(0);
    OLED_W_SCL(1);
    OLED_W_SDA(1);
}
```

图3.4.3.5 IIC程序模块部分

OLED显示字符串，在OLED_ShowString函数中，第一个参数表示起始行的位

置，第二个参数表示起始列的位置，第三个参数表示要显示的字符串，范围可见ASCII码。

```
void OLED_ShowString(uint8_t Line, uint8_t Column, char *String)
{
    uint8_t i;
    for (i = 0; String[i] != '\0'; i++)
    {
        OLED_ShowChar(Line, Column + i, String[i]);
    }
}
```

图3.4.3.6 OLED字符串显示

```
int main(void)
{
    CH32_Init();
    OLED_Init();
    while(1){
        OLED_ShowString(1, 1, "hellow world");
        OLED_ShowString(2, 1, "hellow world");
    }
    return 1;
}
```

图3.4.3.7 主程序

3.4.4 实验结果

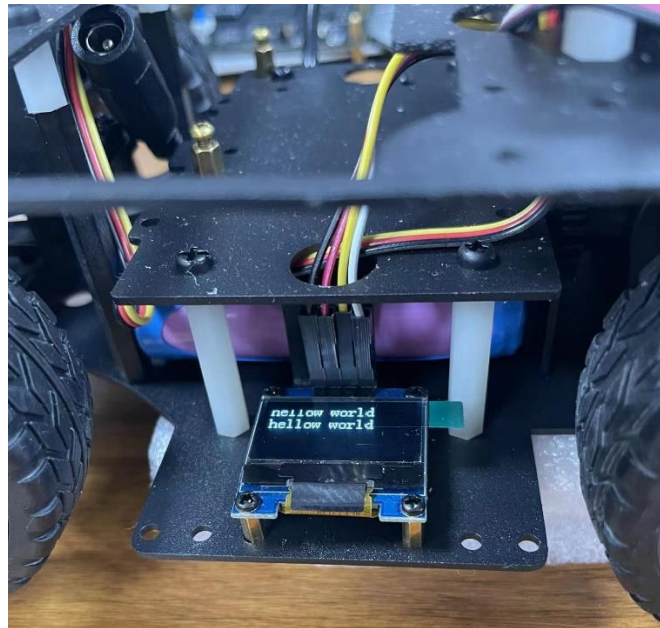


图3.4.4.1 OLED